

RĪGAS VALSTS TEHNIKUMS

DATORIKAS NODAĻA

Izglītības programma: Programmēšana

KVALIFIKĀCIJAS DARBS

**"Tīmekļa sociālā platforma pasākumu veidošanai un
pārvaldībai ar interaktīvu karti"**

Paskaidrojošais raksts 112 lpp.

Audzēknis:

Ņikita Osipovs DP4-4

Prakses vadītājs:

Aleksis Šteinburgs

Nodaļas vadītājs:

Normunds Barbāns

ANOTĀCIJA

Kvalifikācijas darba ietvaros tika izstrādāta tīmekļa sociālā platforma pasākumu veidošanai, meklēšanai un pārvaldībai, izmantojot interaktīvu karti. Sistēma nodrošina iespēju lietotājiem reģistrēties, pārvaldīt savus profilus, izveidot un rediģēt pasākumus, kā arī meklēt un filtrēt notikumus pēc dažādiem kritērijiem. Papildus ir realizēta komentēšana, dalības statusa atzīmēšana, lietotāju sekošana un draudzības pieprasījumi. Sistēmas izstrādē tika izmantotas tādas tehnoloģijas kā PHP 8.2 ar Laravel 12 servera pusē, Next.js 15 klienta pusē, Postgresql datubāze, Leaflet kartes bibliotēka, kā arī Laravel Sanctum autentifikācijai un Laravel Verber paziņojumu apstrādei, Capacitor, lai nodrošinātu saderību ar viedtālruni.

Kvalifikācijas darbs sastāv no ievada, uzdevuma nostādnes, prasību specifikācijas, uzdevuma risināšanas līdzekļu izvēles pamatojuma, programmatūras produkta modelēšanas un projektēšanas apraksta, datu struktūru apraksta, lietotāja ceļveža un testēšanas apraksta. Ievadā ir pamatota sistēmas aktualitāte un nepieciešamība. Uzdevuma nostādnē ir definēti sistēmas galvenie uzdevumi. Prasību specifikācijā ir aprakstīta ieejas un izejas informācija, kā arī funkcionālās un nefunkcionālās prasības. Modelēšanas un projektēšanas sadaļā ir aprakstīta sistēmas arhitektūra un datu modeļi. Datu struktūru aprakstā ir attēlota datubāzes relāciju shēma un tabulu struktūra. Lietotāja ceļvedī ir sniegta informācija par sistēmas lietošanu, bet testēšanas sadaļā ir aprakstīti sistēmas darbības pārbaudes piemēri.

Kvalifikācijas darbs sastāv no 112 lappusēm, kurās ietilpst 33 attēli, 20 tabulas un 1 pielikuma. Pielikums satur programmas pirmkodu fragmentus.

ANOTATION

As part of the qualification work, a web social platform for creating, searching and managing events was developed using an interactive map. The system provides the ability for users to register, manage their profiles, create and edit events, as well as search and filter events by various criteria. In addition, commenting, marking membership status, following users and friend requests have been implemented. The development of the system used technologies such as PHP 8.2 with Laravel 12 on the server side, Next.js 15 on the client side, Postgresql database, Leaflet map library, as well as Laravel Sanctum for authentication and Laravel Reverb for notification processing, Capacitor to ensure compatibility with smartphones.

The qualification work consists of an introduction, task statement, requirements specification, justification for the choice of task solution tools, description of software product modeling and design, description of data structures, user guide and testing description. The introduction justifies the relevance and necessity of the system. The task statement defines the main tasks of the system. The requirements specification describes input and output information, as well as functional and non-functional requirements. The modeling and design section describes the system architecture and data models. The description of data structures depicts the database relational schema and table structure. The user guide provides information on using the system, while the testing section describes examples of system performance testing.

The qualification work consists of 112 pages, which include 33 figures, 20 tables and 1 appendix. The appendix contains fragments of the program source code.

SATURS

IEVADS.....	4
1. UZDEVUMA NOSTĀDNE.....	6
2. PRASĪBU SPECIFIKĀCIJA.....	8
2.1. Ieejas un izejas informācijas apraksts.....	8
2.1.1. Ieejas informācijas apraksts.....	8
2.1.2. Izejas informācijas apraksts.....	8
2.2. Funkcionālās prasības.....	8
2.3. Nefunkcionālās prasības.....	8
3. UZDEVUMA RISINĀŠANAS LĪDZEKĻU IZVĒLES PAMATOJUMS.....	8
4. PROGRAMMATŪRAS PRODUKTA MODELĒŠANA UN PROJEKTĒŠANA.....	8
4.1. Sistēmas struktūras modelis.....	8
4.1.1. Sistēmas arhitektūra.....	8
4.1.2. Sistēmas ER modelis.....	8
4.2. Funkcionālais sistēmas modelis.....	8
4.2.1. Datu plūsmu modelis.....	8
5. DATU STRUKTŪRU APRAKSTS.....	8
6. LIETOTĀJA CEĻVEDIS.....	8
6.1. Sistēmas prasības aparatūrai un programmatūrai.....	8
6.2. Sistēmas instalācija un palaišana.....	8
6.3. Programmas apraksts.....	8
6.4. Programmatūras lietojamības testēšana.....	8
NOBEIGUMS.....	8
INORMĀCIJAS AVOTI.....	8
PIELIKUMI.....	8

IEVADS

Mūsdienās arvien lielāku nozīmi iegūst digitālie risinājumi, kas palīdz lietotājiem ātri un ērti piekļūt aktuālai informācijai. Īpaši svarīga ir informācijas pieejamība par notikumiem un aktivitātēm lietotāja tuvumā, jo tā veicina sabiedrības iesaisti, brīvā laika plānošanu un vietējo kopienu attīstību. Neskatoties uz to, ka pastāv dažādas sociālās platformas un pasākumu publicēšanas vietnes, lielākā daļa no tām nepiedāvā pietiekami ērtu un pārskatāmu veidu, kā lietotājiem atlasīt un apskatīt tuvumā notiekošos pasākumus. Informācija bieži ir izkļaidēta, grūti filtrējama un nepietiekami vizualizēta, kā rezultātā daudzi pasākumi paliek nepamanīti.

Šāda situācija norāda uz nepieciešamību pēc vienota un lietotājam draudzīga risinājuma, kas ļautu efektīvi pārvaldīt un attēlot informāciju par pasākumiem, īpaši koncentrējoties uz konkrētu ģeogrāfisko teritoriju. Lai risinātu šo problēmu, šī kvalifikācijas darba ietvaros tiek izstrādāta tīmekļa lietojumprogramma, kas nodrošina iespēju lietotājiem izveidot un pārlūkot pasākumus, izmantojot interaktīvu karti. Sistēma ļauj pievienot pasākumus ar precīzu atrašanās vietu, kā arī nodrošina vizuāli saprotamu veidu, kā pārskatīt notikumus noteiktā teritorijā.

Papildus pamata funkcionalitātei sistēma piedāvā arī lietotāju savstarpējās mijiedarbības iespējas, piemēram, balsošanu un komentēšanu, kas palīdz novērtēt pasākumu aktualitāti un popularitāti. Tiek ieviesta arī draudzības un abonēšanas funkcionalitāte, kas ļauj lietotājiem sekot citu lietotāju aktivitātēm. Sistēma nodrošina arī privātuma iespējas, ļaujot veidot gan publiskus, gan ierobežotas piekļuves pasākumus, piemēram, tikai draugu lokam.

Tirgus izpētes laikā tika analizēti vairāki esošie risinājumi. Latvijā viens no zināmākajiem piemēriem ir vietne pilsetas.lv, kas apkopo informāciju par dažādiem pasākumiem, tomēr tai trūkst interaktīvas kartes funkcionalitātes. Savukārt starptautiskā mērogā plaši izmantota platforma ir Eventbrite, kas piedāvā plašu pasākumu pārvaldības funkcionalitāti, taču tā galvenokārt orientēta uz lielākiem, komerciāliem pasākumiem un ne vienmēr ir piemērota lokālu, nelielu aktivitāšu atklāšanai konkrētā reģionā.

Veiktās analīzes rezultātā var secināt, ka esošie risinājumi pilnībā neatbilst lietotāju vajadzībām, kuri vēlas ātri un vizuāli uztverami atrast pasākumus savā apkārtnē. Lielākā daļa platformu fokusējas uz vispārēju auditoriju vai konkrētiem pasākumu tipiem, nevis uz lokalizētu un intuitīvu informācijas attēlošanu. Tas rada nepieciešamību pēc specializēta risinājuma, kas būtu pielāgots konkrētai mērķauditorijai un nodrošinātu ērtu lietošanu.

Izstrādājamās sistēmas mērķauditorija ir Latvijas iedzīvotāji, kuri vēlas uzzināt par tuvumā notiekošajiem pasākumiem, kā arī lietotāji, kuri vēlas paši organizēt aktivitātes un piesaistīt

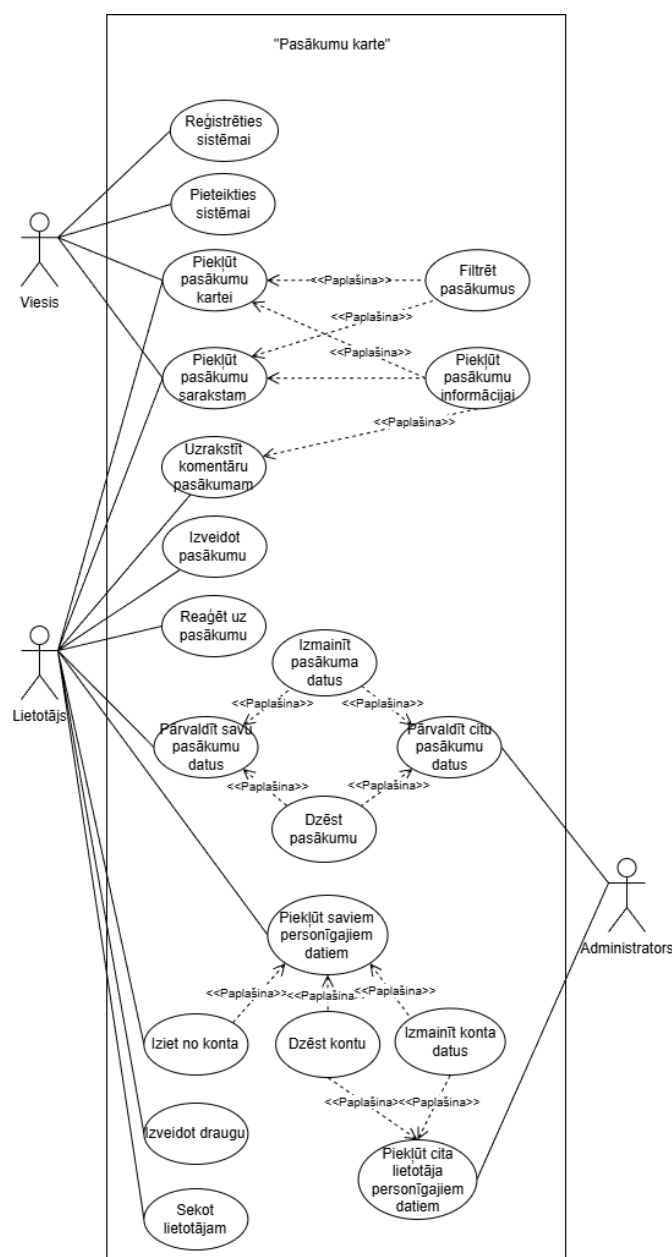
dalībniekus. Sistēma ir piemērota arī tūristiem, kuri vēlas iepazīt Latvijā notiekošos pasākumus un kultūras aktivitātes. Papildus tam nozīmīga lietotāju grupa ir pasākumu organizatori, gan profesionāļi, gan individuālie iniciatori, kuriem nepieciešams vienkāršs un efektīvs rīks savas auditorijas atrašanai.

Izstrādātais risinājums atšķirsies no esošajiem ar to, ka tas būs orientēts tieši uz Latvijas teritoriju un izmantos interaktīvu karti kā galveno informācijas attēlošanas un navigācijas rīku. Tas ļaus lietotājiem intuitīvi atrast pasākumus pēc to atrašanās vietas, kā arī veicinās sociālo mijiedarbību un lokālo kopienu attīstību. Sistēma būs pieejama kā tīmekļa lietojumprogramma, kas padarīs to viegli pieejamu plašam lietotāju lokam.

1. UZDEVUMA NOSTĀDNE

Kvalifikācijas darba uzdevums ir izstrādāt tīmekļa lietojumprogrammu pasākumu meklēšanai un izveidei, kas izmanto interaktīvu karti kā galveno informācijas attēlošanas rīku. Lietotājiem ir jābūt iespējai apskatīt tuvumā notiekošos pasākumus, izveidot savus pasākumus, kā arī mijiedarboties ar citiem lietotājiem, izmantojot komentēšanas, balsošanas un draudzības funkcijas. Sistēmas mērķis ir nodrošināt ērtu un intuitīvu veidu, kā atrast un organizēt pasākumus vienotā platformā.

Sistēmā ir paredzētas šādas galvenās funkcionalitātes (skat. 1.1. att.):



1.1.att. Lietojumgadījumu diagramma

1. Viesim būs iespēja:

- apskatīt pasākumus interaktīvajā kartē un sarakstā;
- filtrēt pasākumus pēc datuma, vietas un kategorijas;
- apskatīt detalizētu informāciju par pasākumu;
- reģistrēties sistēmā un izveidot lietotāja kontu;
- pieteikties sistēmā.

2. Reģistrētam lietotājam būs iespēja:

- darīt visu, ko var darīt viesis;
- izveidot, rediģēt un dzēst savus pasākumus;
- komentēt pasākumus un balsot par tiem;
- sekot citiem lietotājiem un pārvaldīt draudzības pieprasījumus;
- rediģēt savu profila informāciju un dzēst kontu.

3. Administratoram būs iespēja:

- darīt visu, ko var darīt lietotājs;
- pārvaldīt visus pasākumus sistēmā (rediģēt un dzēst);
- pārvaldīt lietotāju kontus.

2. PRASĪBU SPECIFIKĀCIJA

2.1. Ieejas un izejas informācijas apraksts

2.1.1. Ieejas informācijas apraksts

1. Informācija par lietotājiem sastāvēs no šādiem datiem:
 - lietotājvārds - lietotāja unikāls pseidonīms - burtu, ciparu un simbolu teksts ar izmēru līdz 30 rakstzīmēm (piem. Sapog123);
 - E-pasts - lietotāja elektroniskā pasta adrese - burtu teksts ar izmēru līdz 30 rakstzīmēm (piem. kristian@gmail.com);
 - Parole - parole, ar kuru lietotājs autorizējas sistēmā - burtu teksts ar izmēru līdz 30 rakstzīmēm (piem. Parole!123).
2. Informācija par pasākumu sastāvēs no šādiem datiem:
 - nosaukums - pasākumu nosaukums - burtu teksts ar izmēru līdz 50 rakstzīmēm (piem. unicon 2026);
 - platums (latitude) - pasākuma vietas platums kartē - decimālskaitlis ar 6 cipariem aiz komata (piem. 41.40338);
 - garums (longitude) - pasākuma vietas garums kartē - decimālskaitlis ar 6 cipariem aiz komata (piem. 20.17403);
 - adrese - pasākuma vietas nosaukums - burtu teksts ar izmēru līdz 255 rakstzīmēm (piem. brīvības iela 12, rīga);
 - saturs - pasākuma apraksta teksts - teksts ar izmēru līdz 2000 rakstzīmēm. (piem. liels koncerts ar rokmūziku);
 - sākuma datums - pasākuma sākuma datums - datums un laiks (piem. 28.09.2025 10:45);
 - beigu datums - pasākuma beigu datums - datums un laiks (piem. 29.09.2025 15:00).
 - cena - cik maksa biļete - skaitlis (piem. 12.99);
 - pasākuma redzamība - pasākuma redzamības veids - burtu teksts (piem. public, private, friends_only);
 - kategorijas - pasākumam piesaistīto kategoriju identifikatori - skaitļu masīvs (piem. [1, 3, 7]).

3. Informācija par pasākumu reakciju sastāvēs no šādiem datiem:
 - interesē - lietotāja atzīme, ka pasākums interesē - boolean (piem. true);
 - apmeklēšu - lietotāja atzīme, ka lietotājs plāno apmeklēt pasākumu - boolean (piem. true);
 - izveidošanas laiks - laiks kad reakcija tika izveidota - datums un laiks (piem. 29.09.2025 15:00).
4. Informācija par komentāru sastāvēs no šādiem datiem:
 - saturs - komentāra teksts - teksts ar garumu līdz 100 rakstzīmēm (piem. es gribu uz šo koncertu);
 - izveidošanas laiks - laiks kad komentārs tika izveidots - datums un laiks (piem. 29.09.2025 15:00).

2.1.2. Izejas informācijas apraksts

Sistēmā tiks nodrošināta šādas izejas informācijas:

- **Pasākuma kartes attēlojums uz ekrāna.** Galvenais sistēmas rezultāts ir interaktīva karte, kurā tiek attēloti lietotāju izveidotie pasākumi. Katrs pasākums kartē tiek attēlots kā marķieris. Nospiežot uz marķieri, lietotājs redz pasākuma nosaukumu, datumu, aprakstu un organizatora vārdu.
- **Jaunumu e-pasta paziņojums.** Ja lietotājs seko kādam organizatoram, sistēma var ģenerēt paziņojumu uz ekrāna vai e-pastā. Paziņojumā būs virsraksts ("Jauns pasākums tavā pilsētā"), īss apraksts un saite, kas atver pasākuma detalizēto lapu.
- **Kalendāra ieraksta izveide.** Nospiežot uz pogu "Izveidot atgādinājumu kalendārā", lietotājam tiek piedāvāta iespēja pievienot izvēlēto pasākumu Google Kalendāram vai citam izmantotajam kalendāram.
- **Dalīšana ar pasākumu.** Nospiežot uz pogas "Dalīties", pēc kā atveras ierīces kopīgošanas logs, kurā var izvēlēties lietotni vai sociālo tīklu, ar kuru nosūtīt pasākuma saiti.

2.2. Funkcionālās prasības

1. Pieteikšanās sistēmai

2.1.tabula

Pieteikšanās sistēmai

Ievads
Funkcija paredzēta, lai lietotājs varētu pieslēgties sistēmai un izmantot autorizēta lietotāja funkcionalitāti (pasākumu veidošana, komentēšana, draudzība, paziņojumi).
Ievadāti
1. E-pasts - obligāts lauks, jāievada lietotāja reģistrētais e-pasts. 2. Parole - obligāts lauks, jāievada lietotāja parole.
Apstrāde
1. Sistēma pārbauda, vai abi obligātie lauki ir aizpildīti. 2. Sistēma validē e-pasta formātu. 3. Sistēma pārbauda, vai lietotājs ar šādu e-pastu eksistē datubāzē. 4. Sistēma pārbauda paroles atbilstību konkrētajam lietotājam. 5. Ja dati ir korekti, sistēma izveido autorizētu sesiju. 6. Ja notikusi kļūda ievaddatos, tiek paziņota kļūda.
Izvaddati
1. Ja e-pasts un parole ir korekti, lietotājam tiek nodrošināta piekļuve sistēmai. 2. Ja dati nav korekti, piekļuve netiek piešķirta un tiek parādīts kļūdas paziņojums.
Kļūdas paziņojumi
1. Ievadīts nepareizs e-pasts vai parole. 2. Nav aizpildīti obligātie lauki.
Izsaucamās prasības
Var veikt jebkurš viesis ar piekļuvi internetam.

2. Lietotāja reģistrācija

2.2.tabula

Lietotāja reģistrācija

Ievads
Funkcija paredzēta, lai viesis varētu izveidot jaunu lietotāja kontu un turpmāk izmantot sistēmas autorizēto funkcionalitāti.
Ievadāti
1. Lietotājvārds - obligāts lauks, lietotāja unikāls nosaukums sistēmā. 2. E-pasts - obligāts lauks, lietotāja elektroniskā pasta adrese. 3. Parole - obligāts lauks, lietotāja konta parole. 4. Paroles apstiprinājums - obligāts lauks, atkārtoti ievadīta parole.
Apstrāde
1. Funkcija pārbauda, vai visi obligātie lauki ir aizpildīti. 2. Funkcija validē e-pasta formātu. 3. Funkcija pārbauda, vai ievadītais e-pasts jau neeksistē sistēmā. 4. Funkcija pārbauda, vai parole sakrīt ar paroles apstiprinājumu. 5. Ja dati ir korekti, sistēma izveido jaunu lietotāja kontu un saglabā to datubāzē.
Izvaddati
1. Ja reģistrācija ir veiksmīga, lietotāja konts tiek izveidots un lietotājs var pieteikties sistēmā. 2. Ja reģistrācija nav veiksmīga, konts netiek izveidots un tiek parādīti kļūdu paziņojumi.
Kļūdas paziņojumi
1. Nav aizpildīti obligātie lauki. 2. Ievadīts nekorekts e-pasta formāts. 3. E-pasts jau ir reģistrēts sistēmā. 4. Parole un paroles apstiprinājums nesakrīt.
Izsaucamās prasības
Var veikt jebkurš viesis ar piekļuvi internetam.

3. Jauna pasākuma izveide

2.3.tabula

Jauna pasākuma izveide

Ievads
Funkcija paredzēta, lai autorizēts lietotājs varētu izveidot jaunu pasākumu un publicēt to sistēmā izvēlētajā redzamības režīmā.
Ievadāti
1. Pasākuma nosaukums - obligāts lauks. 2. Adrese - obligāts lauks. 3. Koordinātas (platums, garums) - obligāti lauki. 4. Sākuma datums/laiks - obligāts lauks. 5. Apraksts - neobligāts lauks. 6. Beigu datums/laiks - neobligāts lauks. 7. Cena - neobligāts lauks (skaitlis). 8. Redzamība - obligāts lauks (public/private/friends_only). 9. Kategorijas - neobligāts lauks (vairākas izvēles). 10. Fona attēls - neobligāts lauks.
Apstrāde
1. Sistēma pārbauda obligāto lauku aizpildījumu. 2. Sistēma validē datu formātu (datumi, cena, koordinātas, attēla tips/izmērs). 3. Sistēma saglabā adresi un pasākuma datus datubāzē. 4. Sistēma piesaista izvēlētajās kategorijas pasākumam.
Izvaddati
1. Korektas ievades gadījumā pasākums tiek izveidots un saglabāts sistēmā. 2. Nekorektas ievades gadījumā pasākums netiek izveidots un tiek parādītas validācijas kļūdas.
Kļūdas paziņojumi
1. Nav aizpildīti obligātie lauki. 2. Nekorekti aizpildīts viens no laukiem. 3. Nekorekts datu formāts (piem., cena vai datums). 4. Nederīgs attēla formāts vai izmērs.
Izsaucamās prasības
Var veikt tikai autorizēts lietotājs.

4. Pasākuma rediģēšana un dzēšana:

- 4.1. pasākumu rediģēt drīkst tikai pasākuma autors vai administrators;
- 4.2. pasākumu dzēst drīkst tikai pasākuma autors vai administrators;
- 4.3. ja lietotājam nav tiesību rediģēt/dzēst notikumu, sistēmai nevajadzētu nodrošināt piekļuvi šī notikuma rediģēšanai;
- 4.4. pirms dzēšanas lietotājam jāsaņem apstiprinājuma dialogs;
- 4.5. dzēšanas gadījumā ir jādzēš arī ar notikumu saistītie dati;
- 4.6. aizverot pasākuma rediģēšanas lapu, vajadzētu parādīties dialogam par datu zudumu.

5. Pasākumu skatīšana, meklēšana un filtrēšana:

- 5.1. viesiem un autorizētiem lietotājiem jānodrošina piekļuve pasākumu kartei un pasākumu sarakstam;
- 5.2. jānodrošina meklēšana pēc atslēgvārda (piemēram, nosaukuma vai adreses teksta);
- 5.3. jānodrošina filtrēšana pēc kategorijām;
- 5.4. jānodrošina sociālie filtri autorizētiem lietotājiem: tikai draugu pasākumi, tikai sekojamo lietotāju pasākumi;
- 5.5. jānodrošina kārtošana pēc popularitātes, tuvākā datuma, ieinteresēto skaita, un cenas;
- 5.6. noklikšķinot uz pasākuma, jāatver detalizēta pasākuma lapa.

6. Pasākuma piekļuves kontrole pēc redzamības:

- 6.1. publiskie pasākumi jāparāda visiem lietotājiem;
- 6.2. "tikai draugiem" pasākumi jāparāda tikai autora draugiem;
- 6.3. privātie pasākumi var piekļūt tikai ar saiti;
- 6.4. ja lietotājam nav piekļuves pasākumam, jāatgriež kļūdu.

7. Lietotāju mijiedarbība ar pasākumiem:

- 7.1. autorizētam lietotājam jābūt iespējai atzīmēt pasākumu kā "interesē";
- 7.2. autorizētam lietotājam jābūt iespējai atzīmēt pasākumu kā "apmeklēšu";
- 7.3. lietotājam jābūt iespējai noņemt abas atzīmes;
- 7.4. pasākuma detalizācijā jāattēlo ieinteresēto un apmeklētāju skaits;
- 7.5. pasākuma detalizācijas lapā jānodrošina poga "dalīties";
- 7.6. jānodrošina poga pasākuma pievienošanai kalendārā.

8. Komentāru pārvaldība:

- 8.1. autorizētam lietotājam jābūt iespējai pievienot komentāru pasākumam;
- 8.2. ja komentārs neatbilst validācijai (piemēram, pārāk īss/tukšs), komentāru nedrīkst saglabāt;
- 8.3. lietotājam jābūt iespējai rediģēt tikai savus komentārus;
- 8.4. lietotājam jābūt iespējai dzēst tikai savus komentārus;
- 8.5. pasākuma autoram jāsaņem paziņojums par jaunu komentāru pie viņa pasākuma.

9. Lietotāja profils un savstarpējā mijiedarbība:

- 9.1. jānodrošina profila apskate ar statistiku (pasākumu skaits, sekotāji, draugi);
- 9.2. autorizētam lietotājam jābūt iespējai sekot citam lietotājam un atcelt sekošanu;
- 9.3. autorizētam lietotājam jābūt iespējai nosūtīt draudzības pieprasījumu citam lietotājam;
- 9.4. autorizētam lietotājam jābūt iespējai pieņemt vai noraidīt saņemto draudzības pieprasījumu;
- 9.5. autorizētam lietotājam jābūt iespējai noņemt draudzību.

10. Cilvēku saraksts un meklēšana:

- 10.1. lietotājam jābūt pieejamai cilvēku (lietotāju) saraksta lapai;
- 10.2. jānodrošina lietotāju meklēšana pēc vārda vai e-pasta;
- 10.3. lietotāju sarakstā nedrīkst tikt attēlots pats autorizētais lietotājs;
- 10.4. no cilvēku saraksta jābūt iespējai atvērt izvēlētā lietotāja profilu.

11. Paziņojumu sistēma:

- 11.1. autorizētam lietotājam jābūt pieejamam paziņojumu sarakstam;
- 11.2. jānodrošina paziņojuma atzīmēšana kā izlasītam;
- 11.3. jānodrošina visu paziņojumu atzīmēšana kā izlasītiem;
- 11.4. jānodrošina viena paziņojuma un visu paziņojumu dzēšana;
- 11.5. jānodrošina paziņojumu preferenču pārvaldība pa tiem (lietotne/e-pasts).

12. Administratora lietotāju pārvaldība:

- 12.1. administratoram jābūt pieejamai lietotāju pārvaldības sadaļai ar meklēšanu;
- 12.2. administratoram jābūt iespējai rediģēt lietotāja vārdu, e-pastu un lomu;
- 12.3. administratoram jābūt iespējai dzēst citu lietotāju kontus;
- 12.4. administratoram nedrīkst būt iespējai noņemt pašam sev administratora lomu.

2.3. Nefunkcionālās prasības

1. Sistēmas saskarnes valodai ir jābūt latviešu valodai.
2. Jānodrošina tas, ka tīmekļa lietojumprogramma automātiski pielāgojas un saglabā ērtu lietojamību dažādos ekrāna izmēros.
3. Sistēma ir jābūt saderīgai ar populārākajiem tīmekļa pārlūkiem, piemēram, Chrome, Firefox, un Safari.
4. Dizainam ir jābūt baltos un zilos toņos.
5. Sistēmai jābūt drošai un nedrīkst pakļaut lietotājam nevajadzīgus datus.
6. Sistēmai jāsniedz lietotājam informatīvus paziņojumus par veiksmīgiem vai neveiksmīgiem procesiem, nodrošinot skaidru lietojamību
7. Pasākumu kartes skatam jābūt ērti lietojamam, ar iespēju tuvināt/attālināt un atlasīt pasākumus pēc filtriem.
8. Notikumu aprakstiem jāizmanto bagātinātā teksta redaktora funkcijas, lai nodrošinātu labāku lietotāja pieredzi (piemēram, galvenes līmeņi, treknraksts, slīpraksts).
9. Lietotāja pieslēgšanās formai jābūt izveidotai pēc noteiktā dizaina (skat. 2.1.att.).

The diagram shows a login form with the title "Pieslēgšanās" centered at the top. Below the title are two input fields. The first field is labeled "e-pasts" and the second field is labeled "parole". Below these fields is a button labeled "Ienākt".

2. 1.att. Lietotāja pieslēgšanas formas skice

10. Lietotāja profila rediģēšanas formai jābūt vizuāli pārskatāmai un jāatbilst izstrādātajai skicei (skat. 2.2.att.).

The sketch shows a user profile editing form with the following elements:

- Title:** Rediģēt manu profilu
- Profile Picture:** A circular placeholder labeled "Attēls".
- Upload Button:** A button labeled "Augšupielādēt attēlu" next to the profile picture.
- Username Field:** A text input field labeled "lietotājs vārds".
- Password Field:** A text input field labeled "parole".
- Confirm Password Field:** A text input field labeled "apstiprināt paroli".
- Save Button:** A button labeled "Saglabāt" at the bottom center.

2.2.att. Lietotāja profila rediģēšanas formas skice

3. UZDEVUMA RISINĀŠANAS LĪDZEKĻU IZVĒLES PAMATOJUMS

Projekta izstrādei tiek izmantots vairāku tehnoloģiju un rīku kopums:

Laravel 12 ietvars ir izvēlēts servera puses funkcionalitātes izstrādei, jo tas piedāvā stabilu arhitektūru, plašu iebūvēto funkciju klāstu un augstu drošības līmeni. Izvēlinosaka arī personīgā pieredze, strādājot ar šo sistēmu iepriekšējā mācību gadā. Laravel arī ir plaša un saprotama dokumentācija.

Next.js 15 tiek izmantots lietotāja saskarnes izstrādei, jo tas apvieno React darbības principus ar servera puses renderēšanas iespējām. Next.js izvēli pamato arī vēlme padziļināt zināšanas par mūsdienu JavaScript tehnoloģijām un izmantot rīku, kas pašlaik ir viens no populārākajiem..

Leaflet kalpo kā galvenais kartes risinājums platformā. Leaflet ir viena no populārākajām atvērtā koda JavaScript bibliotēkām interaktīvu karšu izveidei, kas nodrošina ātru darbību un elastīgu pielāgošanu.

Docker tiek izmantots izstrādes vides un lietojumprogrammu konteinerizācijai. Šī tehnoloģija nodrošina izolētu un reproducējamu vidi, kas novērš konfigurācijas problēmas un atvieglo sistēmas instalēšanu dažādās ierīcēs.

Datu glabāšanai tika izvēlēta **Postgresql** datubāzes pārvaldības sistēma, pamatojoties uz iepriekšējo pieredzi ar šo tehnoloģiju, ka arī tā ir uzticama, ātra un efektīva relāciju datubāze, ko plaši izmanto gan izglītības, gan profesionālā vidē.

Capacitor tiek izmantots kā rīks tīmekļa lietotnes pārveidošanai par mobilo lietotni. Capacitor ir moderns un plaši izmantots risinājums, kas ļauj izstrādātājiem izmantot vienotu kodu gan tīmekļa, gan mobilajām platformām. Capacitor garantē efektīvu lietotnes darbību mobilajās ierīcēs un samazina izstrādes sarežģītību.

Laravel Reverb lietots paziņojumu un reāllaika komunikācijas nodrošināšanai.

Obsidian tiek izmantots kā rīks piezīmju un tehniskās projekta informācijas organizēšana.

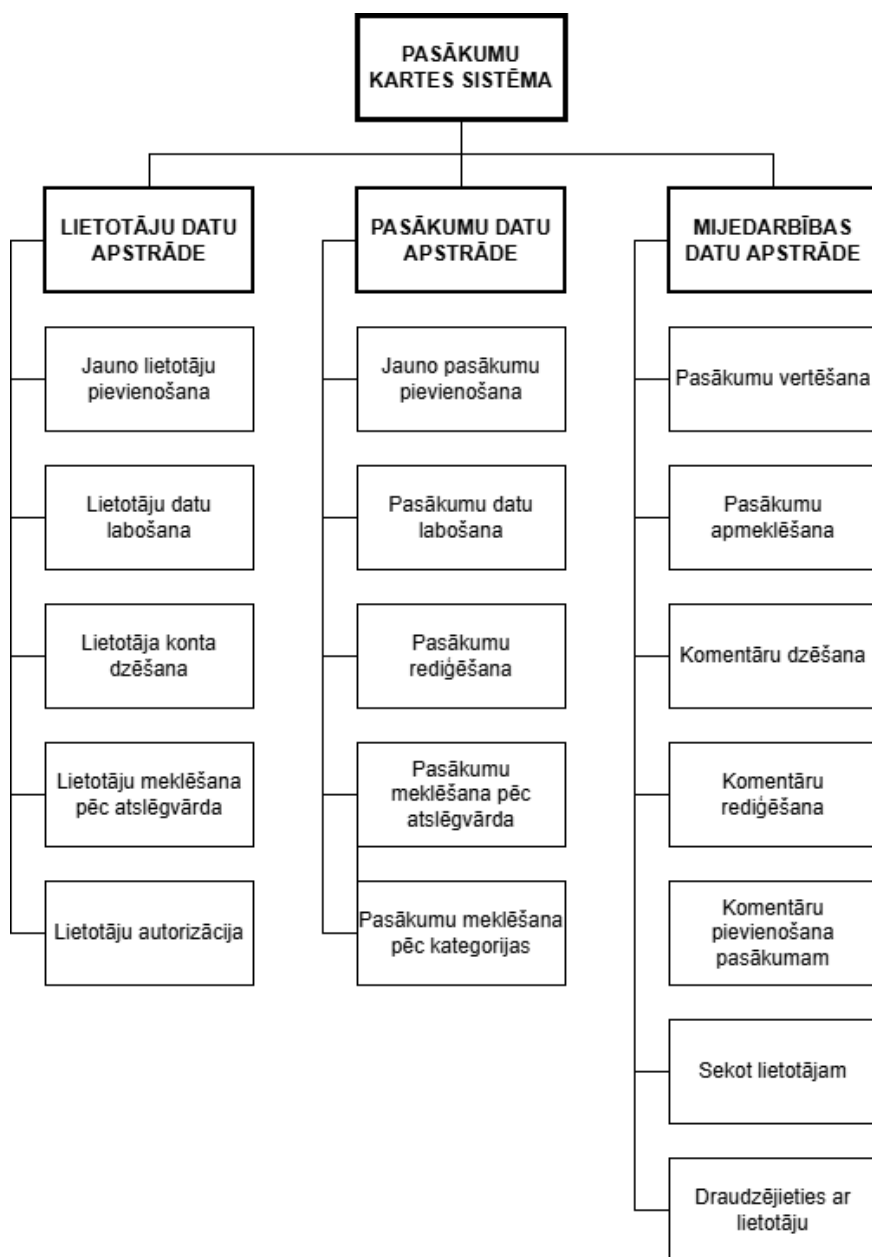
Microsoft 365 tiek izmantots oficiālas projekta dokumentācijas sagatavošanai un formatēšanai.

4. PROGRAMMATŪRAS PRODUKTA MODELĒŠANA UN PROJEKTĒŠANA

4.1. Sistēmas struktūras modelis

4.1.1. Sistēmas arhitektūra

Sistēma tiks veidota no četrām apakšsistēmām (skat. 4.1.att): lietotāju datu apstrādes apakšsistēmas, pasākumu datu apstrādes apakšsistēmas un mijiedarbības datu apstrādes apakšsistēmas.



4.1.att. Funkcionālās dekompozīcijas diagramma

Lietotāju datu apstrāde. Šī apakšsistēma nodrošina lietotāju kontu pārvaldību un autorizāciju sistēmā.

Tajā ietilpst šādi moduļi:

- jauno lietotāju pievienošana - nodrošina jaunu lietotāju reģistrāciju sistēmā;
- lietotāju datu labošana - nodrošina iespēju lietotājiem rediģēt savu profila informāciju; lietotāja konta dzēšana - nodrošina lietotāja konta dzēšanu;
- lietotāju meklēšana pēc atslēgvārda - nodrošina lietotāju meklēšanu pēc noteiktiem kritērijiem;
- lietotāju autorizācija - nodrošina lietotāju pieteikšanos sistēmā.

Pasākumu datu apstrāde. Šī apakšsistēma nodrošina pasākumu pārvaldību sistēmā.

Tajā ietilpst šādi moduļi:

- jauno pasākumu pievienošana - nodrošina jaunu pasākumu izveidi;
- pasākumu datu labošana - nodrošina pasākumu informācijas rediģēšanu;
- pasākumu dzēšana - nodrošina pasākumu dzēšanu;
- pasākumu meklēšana pēc atslēgvārda - nodrošina pasākumu meklēšanu;
- pasākumu meklēšana pēc kategorijas - nodrošina pasākumu filtrēšanu pēc kategorijas;

Mijiedarbības datu apstrāde. Šī apakšsistēma nodrošina lietotāju savstarpējo mijiedarbību un aktivitātes sistēmā.

Tajā ietilpst šādi moduļi:

- pasākumu vērtēšana - nodrošina iespēju lietotājiem novērtēt pasākumus;
- pasākumu apmeklēšana - nodrošina lietotājiem atzīmēt, ka viņi apmeklēs pasākumu;
- komentāru pievienošana pasākumam - nodrošina komentāru izveidi;
- komentāru rediģēšana - nodrošina komentāru informācijas rediģēšanu;
- komentāru dzēšana - nodrošina komentāru informācijas dzēšanu;
- sekot lietotājam - nodrošina iespēju sekot citiem lietotājiem;
- draudzēties ar lietotāju - nodrošina draudzības izveidi starp lietotājiem.

4.1.2. Sistēmas ER modelis

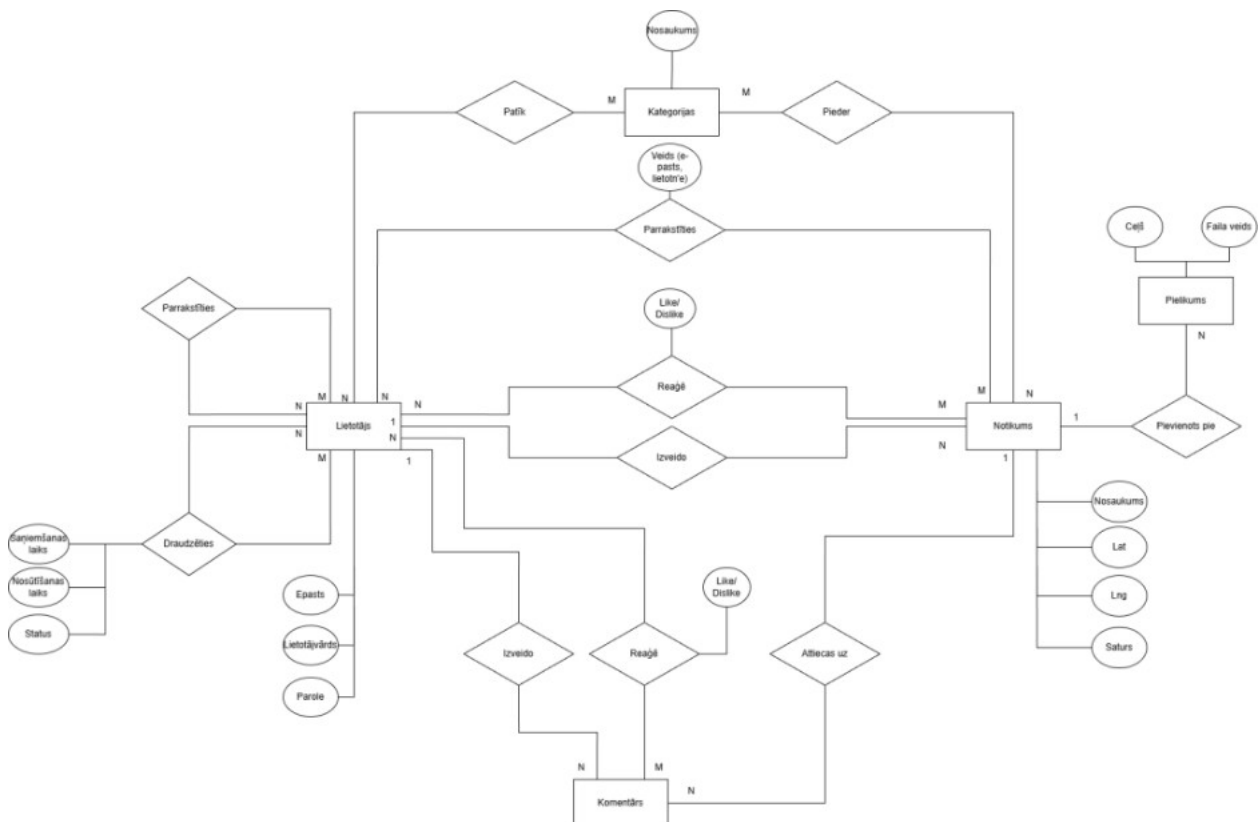
Datu bāzes projektēšanā datu kopu un saišu starp tām attēlošanai tika lietota realitāšu saišu diagramma, kas sastāv no divu veidu objektiem - entītēm un relācijām. ER diagramma (sk. 4.2. att) sastāv no 6 entītijām:

- **lietotājs** - satur informāciju par sistēmas lietotājiem, piemēram, lietotājevārdu, e-pastu, paroli, profila attēlu un lomu;
- **notikums** - satur informāciju par sistēmā izveidotajiem pasākumiem, piemēram, nosaukumu, aprakstu, atrašanās vietu, koordinātēm, redzamību un adresi;
- **komentārs** - satur lietotāju izveidotos komentārus, piemēram, komentāra tekstu;
- **kategorija** - satur notikumu kategorijas, piemēram, kategorijas nosaukumu;
- **paziņojumi** - satur informāciju par lietotājiem nosūtītajiem paziņojumiem, piemēram, tekstu, tipu un nosūtīšanas laiku;
- **paziņojumu preferences** - satur lietotāju izvēlētos paziņojumu iestatījumus, piemēram, paziņojumu veidu, lietotni un e-pastu.

ER-modelis demonstrē entītiju savstarpējo saistību:

- starp "Lietotājs" un "Notikums" pastāv relācija "Izveido" viens pret daudziem (1-N), jo viens lietotājs var izveidot vairākus notikumus, bet katru notikumu izveido tikai viens lietotājs;
- starp "Lietotājs" un "Notikums" pastāv relācija "Interesē" daudzi pret daudziem (M-N), jo viens lietotājs var interesēties par vairākiem notikumiem un viens notikums var interesēt vairākus lietotājus;
- starp "Lietotājs" un "Notikums" pastāv relācija "Apmeklē" daudzi pret daudziem (M-N), jo lietotājs var apmeklēt vairākus notikumus un vienu notikumu var apmeklēt daudzi lietotāji;
- starp "Lietotājs" un "Komentārs" pastāv relācija "Izveido" viens pret daudziem (1-N), jo viens lietotājs var izveidot vairākus komentārus, bet katru komentāru izveido tikai viens lietotājs;
- starp "Komentārs" un "Notikums" pastāv relācija "Attiecas uz" daudzi pret vienu (N-1), jo vairāki komentāri var attiekties uz vienu notikumu, bet katrs komentārs attiecas tikai uz vienu notikumu;

- starp "Notikums" un "Kategorijas" pastāv relācija "Pieder" daudzi pret vienu (N-1), jo vairāki notikumi var piederēt vienai kategorijai, bet katrs notikums pieder tikai vienai kategorijai;
- starp "Lietotājs" un "Lietotājs" pastāv relācija "Pieprasa draudzību" daudzi pret daudziem (M-N), jo lietotāji var sūtīt draudzības pieprasījumus viens otram;
- starp "Lietotājs" un "Lietotājs" pastāv relācija "Parakstīties" daudzi pret daudziem (M-N), jo lietotāji var sekot citiem lietotājiem un vienam lietotājam var būt vairāki sekotāji;
- starp "Lietotājs" un "Paziņojumu preferences" pastāv relācija "Pieder kam" viens pret daudziem (1-N), jo vienam lietotājam var būt vairākas paziņojumu preferences.
- starp "Paziņojumi" un "Lietotājs" pastāv relācija "Paziņot kam" daudzi pret vienu (N-1), jo vairāki paziņojumi var būt adresēti vienam lietotājam, bet katrs paziņojums ir saistīts ar vienu lietotāju.



4.2..att. ER modelis

4.2. Funkcionālais sistēmas modelis

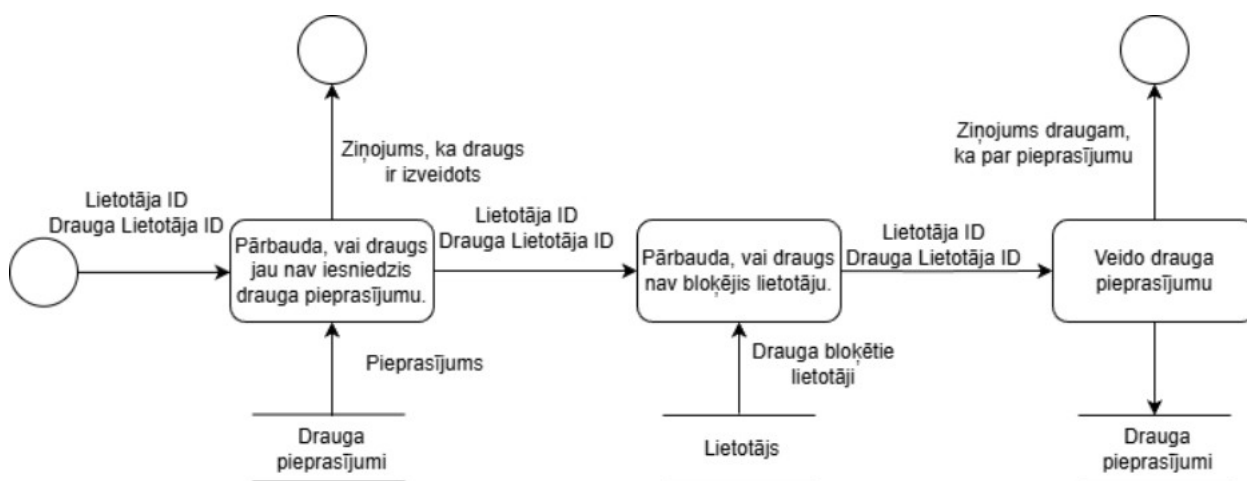
4.2.1. Datu plūsmu modelis

1. Draugu pieprasījuma veidošana (skat. 4.3. att)

Lietotājs ievada savu lietotāja ID un drauga lietotāja ID. Sistēma pārbauda, vai draugs jau nav iesniedzis drauga pieprasījumu, izmantojot datu krātuvi Drauga pieprasījumi. Ja pieprasījums jau eksistē, tiek izveidota draudzība un lietotājam tiek izvadīts ziņojums, ka draugs ir pievienots.

Ja pieprasījums neeksistē, sistēma pārbauda, vai draugs nav bloķējis lietotāju, izmantojot datu krātuvi Lietotāji. Ja lietotājs ir bloķēts, tiek izvadīts kļūdas ziņojums un darbība netiek turpināta.

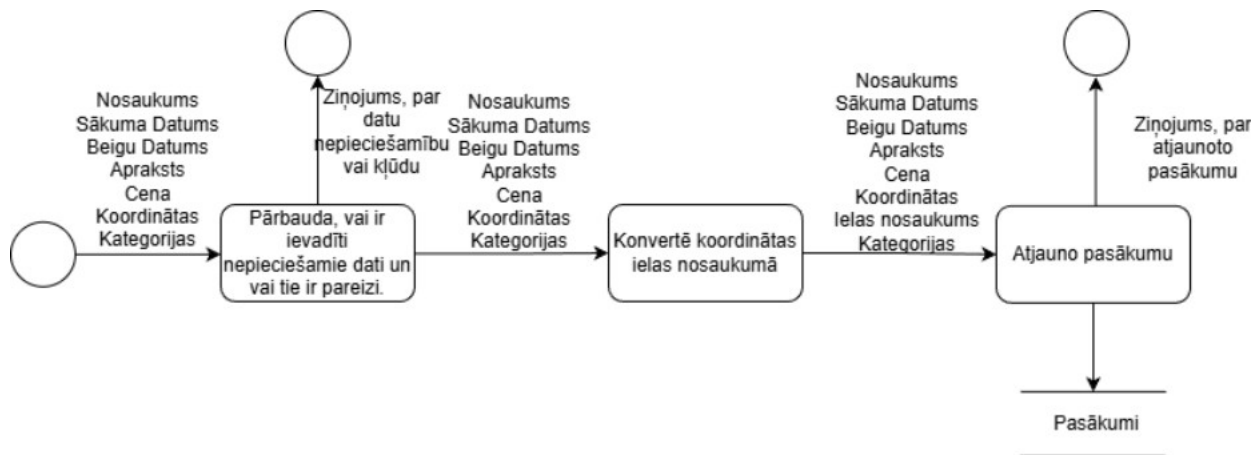
Ja visi nosacījumi ir izpildīti, sistēma izveido jaunu drauga pieprasījumu un saglabā to datu krātuvē Drauga pieprasījumi. Pēc tam draugam tiek nosūtīts paziņojums par jauno pieprasījumu.



4.3.att. Draugu pieprasījuma veidošanas moduļa datu plūsmas diagramma

2. Pasakuma atjauninājums (skat. 4.4. att)

Lietotājs ievada pasākuma informāciju. Sistēma pārbauda datu korektumu. Lietotāju ērtībām koordinātas jāpārveido ielu nosaukumos.. Ja dati ir pareizi, tie tiek saglabāti datu krātuvē "Pasākumi" un ziņojumu par atjaunošanu tiek nosūtīts.

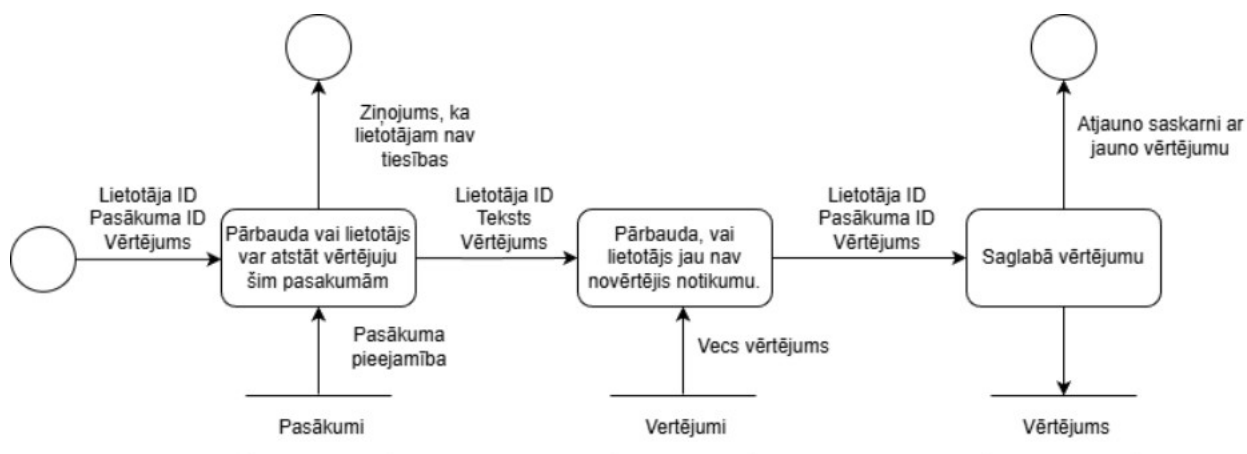


4.4.att. Pasakuma atjauninājuma moduļa datu plūsmas diagramma

3. Pasakumu vērtēšana (skat. 4.5. att)

Lietotājs ievada lietotāja ID, pasākuma ID un vērtējumu. Sistēma pārbauda, vai lietotājam ir tiesības atstāt vērtējumu šim pasākumam, izmantojot datu krātuvi "Pasākumi". Ja lietotājam nav tiesību (piemēram, nav apmeklējis pasākumu), tiek izvadīts kļūdas ziņojums.

Ja lietotājam ir tiesības, sistēma saglabā vērtējumu datu krātuvē "Vērtējumi". Pēc tam sistēma atjauno lietotāja saskarni, lai attēlotu jauno vērtējumu.



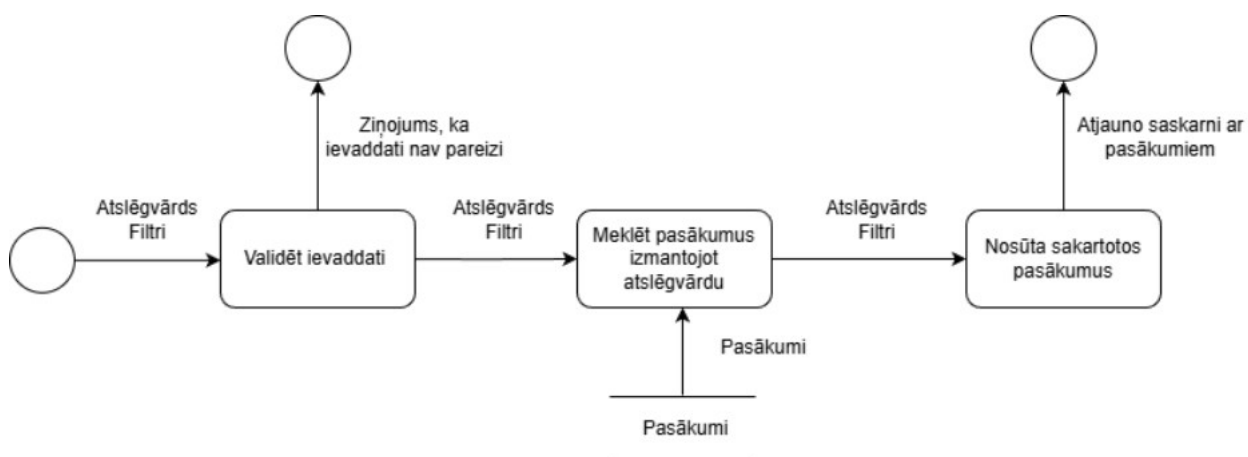
4.5.att. Pasakumu vērtēšanas moduļa datu plūsmas diagramma

4. Pasakumu meklēšana (skat. 4.6. att)

Lietotājs ievada atslēgvārdu un izvēlas filtrus. Sistēma validē ievaddatus, pārbaudot vai atslēgvārds nav tukšs un vai filtri ir korekti. Ja ievaddati nav pareizi, tiek izvadīts kļūdas ziņojums.

Ja ievaddati ir korekti, sistēma meklē pasākumus, izmantojot atslēgvārdu datu krātuvē "Pasākumi". Pēc tam sistēma sakārto atrastos pasākumus atbilstoši izvēlētajiem filtriem.

Beigās sistēma atjauno lietotāja saskarni un attēlo atrastos un sakārtotos pasākumus.



4.6.att. Pasakumu meklēšana moduļa datu plūsmas diagramma

5. DATU STRUKTŪRU APRAKSTS

Datubāzes projektēšanas rezultātā tika izveidotas vairākas tabulas, un tika definēta to relācija jeb saistība (skat. 5.1. att.).

Datubāze sastāv no 14 tabulām, un satur informāciju par sistēmas lietotājiem, lietotāju savstarpējām attiecībām, pasākumiem, pasākumu kategorijām, adresēm, redzamības veidiem, komentāriem, paziņojumiem, paziņojumu iestatījumiem.

- tabula "Users" glabā informāciju par reģistrētiem sistēmas lietotājiem;
- tabula "Events" glabā informāciju par sistēmā izveidotajiem pasākumiem;
- tabula "Comments" glabā lietotāju komentārus pie notikumiem;
- tabula "Going" glabā informāciju par lietotājiem, kuri plāno apmeklēt pasākumus;
- tabula "Interested" glabā informāciju par lietotājiem, kuri ir ieinteresēti pasākumos;
- tabula "Notifications" glabā lietotājiem nosūtītos paziņojumus;
- tabula "NotificationPreferences" glabā lietotāju paziņojumu iestatījumus;
- tabula "Followers" glabā informāciju par lietotāju abonēšanu;
- tabula "FriendshipRequests" glabā lietotāju draudzības pieprasījumus;
- tabula "Friends" glabā informāciju par apstiprinātām draudzībām;
- tabula "EventCategories" glabā pasākumu kategorijas;
- tabula "EventCategoryMapping" sasaista pasākumus ar kategorijām;
- tabula "EventVisibilities" glabā pasākumu redzamības veidus;
- tabula "Addresses" glabā pasākumu atrašanās vietas datus.

Tabula "Users" satur reģistrētu lietotāju datus: lietotāja unikālo identifikatoru, vārdu, e-pasta adresi, paroli, lomu, profila attēla ceļu, e-pasta apstiprināšanas datumu, kā arī ieraksta izveides un atjaunošanas datumus.

Tabulas "Users" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	Id	INT	-	Lietotāja identifikators
2.	Name	VARCHAR	255	Lietotāja vārds
3.	Email	VARCHAR	255	Lietotāja e-pasta adrese
4.	Password	VARCHAR	255	Lietotāja parole
5.	Role	VARCHAR	20	Lietotāja loma sistēmā
6.	AvatarPath	VARCHAR	255	Lietotāja profila attēla ceļš
7.	EmailVerifiedAt	DATE	-	E-pasta apstiprināšanas datums
8.	UpdatedAt	DATE	-	Ieraksta pēdējās atjaunošanas datums
9.	CreatedAt	DATE	-	Ieraksta izveides datums

Tabula "Events" satur sistēmā izveidoto pasākuma datus: pasākuma nosaukumu, aprakstu, sākuma un beigu datumu, cenu, attēla ceļu, redzamības veidu, adresi un lietotāju, kurš izveidoja notikumu.

Tabulas "Events" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	Id	INT	-	Pasākuma identifikators
2.	Title	VARCHAR	255	Pasākuma nosaukums
3.	Description	TEXT	-	Pasākuma apraksts
4.	StartDate	DATE	-	Pasākuma sākuma datums
5.	EndDate	DATE	8,2	Pasākuma beigu datums
6.	Price	DECIMAL	255	Pasākuma cena
7.	BackgroundImagePath	VARCHAR	-	Pasākuma attēla ceļš
8.	EventVisibilityId	INT	-	Pasākuma redzamības veida identifikators
9.	AddressId	INT	-	Pasākuma adreses identifikators
10.	UserId	INT	-	Lietotāja identifikators, kurš izveidoja notikumu
11.	UpdatedAt	DATE	-	Ieraksta pēdējās atjaunošanas datums
12.	CreatedAt	DATE	-	Ieraksta izveides datums

Tabula "Comments" satur sistēmā izveidoto komentāru datus: komentāra tekstu, lietotāja identifikatoru, kurš izveidoja komentāru, pasākuma identifikatoru, pie kura komentārs tika izveidots.

5.3.tabula

Tabulas "Comments" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	Id	INT	-	Komentāra identifikators
2.	Text	VARCHAR	500	Komentāra teksts
3.	UserId	INT	-	Lietotāja identifikators, kurš izveidoja komentāru
4.	EventId	INT	-	Pasākuma identifikators, pie kura komentārs tika izveidots
5.	UpdatedAt	DATE	-	Ieraksta pēdējās atjaunošanas datums
6.	CreatedAt	DATE	-	Ieraksta izveides datums

Tabula "Going" satur informāciju par notikumiem, kurus lietotāji plāno apmeklēt. Šī tabula realizē daudzi pret daudziem saiti starp tabulām "Users" un "Events".

5.4.tabula

Tabulas "Going" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	Id	INT	-	Ieraksta identifikators
2.	UserId	INT	-	Lietotāja identifikators, kurš plāno apmeklēt pasākumu
3.	EventId	INT	-	Pasākuma identifikators, kuru lietotājs plāno apmeklēt
4.	UpdatedAt	DATE	-	Ieraksta pēdējās atjaunošanas datums
5.	CreatedAt	DATE	-	Ieraksta izveides datums

Tabula "Interested" satur informāciju par notikumiem, par kuriem lietotāji ir ieinteresēti. Šī tabula realizē daudzi pret daudziem saiti starp tabulām "Users" un "Events".

5.5.tabula

Tabulas "Interested" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	Id	INT	-	Ieraksta identifikators
2.	UserId	INT	-	Lietotāja identifikators
3.	EventId	INT	-	Pasākuma identifikators, kurš lietotāja interesē
4.	UpdatedAt	DATE	-	Ieraksta pēdējās atjaunošanas datums
5.	CreatedAt	DATE	-	Ieraksta izveides datums

Tabula "Notifications" satur lietotājiem nosūtītos sistēmas paziņojumus. Katrs paziņojums ir piesaistīts vienam lietotājam.

5.6.tabula

Tabulas "Notifications" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	Id	INT	-	Paziņojuma identifikators
2.	Type	VARCHAR	255	Paziņojuma tips
3.	Data	Text	-	Paziņojuma saturs
4.	ReadAt	DATE	-	Paziņojuma izlasīšanas datums
5.	UserId	INT	-	Lietotāja identifikators, kuram paziņojums ir paredzēts
6.	UpdatedAt	DATE	-	Ieraksta pēdējās atjaunošanas datums
7.	CreatedAt	DATE	-	Ieraksta izveides datums

Tabula "NotificationPreferences" satur lietotāja paziņojumu iestatījumus, piemēram, vai konkrēta tipa paziņojumi ir ieslēgti lietotnē vai e-pastā.

5.7.tabula

Tabulas "NotificationPreferences" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	Id	INT	-	Ieraksta identifikators
2.	Type	VARCHAR	255	Paziņojuma tips
3.	InAppEnabled	BOOLEAN	-	Norāda, vai paziņojumi lietotnē ir ieslēgti
4.	EmailEnabled	BOOLEAN	-	Norāda, vai paziņojumi e-pastā ir ieslēgti
5.	UserId	INT	-	Lietotāja identifikators
6.	UpdatedAt	DATE	-	Ieraksta pēdējās atjaunošanas datums
7.	CreatedAt	DATE	-	Ieraksta izveides datums

Tabula "Followers" satur informāciju par lietotāju abonēšanu. Viens lietotājs var sekot vairākiem citiem lietotājiem, un vienam lietotājam var būt vairāki sekotāji.

5.8.tabula

Tabulas "NotificationPreferences" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	Id	INT	-	Ieraksta identifikators
2.	FollowerId	INT	255	Lietotāja identifikators, kurš seko
3.	TargetId	INT	-	Lietotāja identifikators, kuram seko
4.	UpdatedAt	DATE	-	Ieraksta pēdējās atjaunošanas datums
5.	CreatedAt	DATE	-	Ieraksta izveides datums

Tabula "FriendshipRequests" satur lietotāju draudzības pieprasījumus. Tajā tiek glabāts pieprasījuma sūtītājs, saņēmējs un pieprasījuma statuss.

5.9.tabula

Tabulas "FriendshipRequests" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	Id	INT	-	Pieprasījuma identifikators
2.	Status	INT	255	Draudzības pieprasījuma statuss
3.	UserId	INT	-	Lietotāja identifikators, kurš nosūtīja pieprasījumu
4.	ReceiverId	INT	-	Lietotāja identifikators, kurš saņēma pieprasījumu
5.	UpdatedAt	DATE	-	Ieraksta pēdējās atjaunošanas datums
6.	CreatedAt	DATE	-	Ieraksta izveides datums

Tabula "Friends" satur informāciju par apstiprinātām draudzībām starp lietotājiem.

5.10.tabula

Tabulas "Friends" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	Id	INT	-	Ieraksta identifikators
2.	User1Id	INT	-	Pirmā lietotāja identifikators
3.	User2Id	INT	-	Otrā lietotāja identifikators
4.	UpdatedAt	DATE	-	Ieraksta pēdējās atjaunošanas datums
5.	CreatedAt	DATE	-	Ieraksta izveides datums

Tabula "EventCategories" satur notikumu kategorijas. Kategorijas tiek izmantotas, lai notikumus būtu iespējams grupēt un filtrēt.

5.11.tabula

Tabulas "EventCategories" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	Id	INT	-	Kategorijas identifikators
2.	Name	VARCHAR	255	Kategorijas nosaukums
3.	UpdatedAt	DATE	-	Ieraksta pēdējās atjaunošanas datums
4.	CreatedAt	DATE	-	Ieraksta izveides datums

Tabula "EventCategoryMapping" ir starptabula, kas sasaista notikumus ar kategorijām. Tā realizē daudzi pret daudziem saiti starp tabulām "Events" un "EventCategories".

5.12.tabula

Tabulas "EventCategoryMapping" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	Id	INT	-	Ieraksts identifikators
2.	EventId	INT	-	Notikuma identifikators
3.	CategoryId	INT	-	Kategorijas identifikators

Tabula "EventVisibilities" satur notikumu redzamības veidus. Tā ļauj noteikt, vai notikums ir publisks vai pieejams tikai noteiktai lietotāju grupai.

5.13.tabula

Tabulas "EventVisibilities" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	Id	INT	-	Redzamības veida identifikators
2.	Name	VARCHAR	20	Redzamības veida nosaukums

Tabula "Addresses" satur notikumu atrašanās vietas datus. Tā glabā adreses nosaukumu un koordinātes, kas nepieciešamas notikumu attēlošanai kartē.

5.14.tabula

Tabulas "Addresses" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	Id	INT	-	Adreses veida identifikators
2.	Lat	DOUBLE	-	Atrašanās vietas platuma koordināte
3.	Lng	DOUBLE	-	Atrašanās vietas garuma koordināte

6. LIETOTĀJA CEĻVEDIS

6.1. Sistēmas prasības aparatūrai un programmatūrai

Pasākumu pārvaldības sistēma ir tīmekļa lietotne, kas ir piemērota ērtai lietošanai lielākajā daļā ierīču, ja tās atbilst sekojošām prasībām:

- Operētājsistēma:
 - Windows 10 vai jaunāka versija;
 - Android ierīce;
 - jebkura moderna Linux versija.
- Interneta Pārlūks:
 - Google Chrome 89 vai jaunāka versija;
 - Mozilla Firefox 86 vai jaunāka versija;
 - Safari 14 vai jaunāka versija;
 - Microsoft Edge 89 vai jaunāka versija.

6.2. Sistēmas instalācija un palaišana

Lai instalētu un palaistu sistēmu lietotājam vispirms ir jāparupējās par sekojoša programmnodrošinājuma instalēšanu uz izmantojamās ierīces:

Visual Studio Code vai cits IDE

PHP 8.2+

Composer

Node.js (LTS)

Postgresql vai citi.

Git

1. Projekta klonēšana no GitHub:
 - 1.1. Atveriet terminālu un izpildiet komandu Git Clone;
2. Nepieciešamo rīku instalēšana un konfigurēšana:
 - 2.1. Ieejiet projekta "backend" mapē;
 - 2.2. Palaidiet komandu: composer install;
 - 2.3. Palaidiet komandu: php artisan key:generate;
 - 2.4. Konfigurējiet backend .env failu;
 - 2.5. Ieejiet projekta "frontend" mapē;
 - 2.6. Palaidiet komandu: npm install;
 - 2.7. Konfigurējiet frontend.env.local failu;
3. Datubāzes konfigurācija:
 - 3.1. Ieejiet projekta "backend" mapē;
 - 3.2. Palaidiet datubāzes migrācijas: php artisan migrate;
 - 3.3. Palaidiet komandu: php artisan db:seed;
 - 3.4. Palaidiet komandu: php artisan storage:link.
4. Projekta palaišana lokālajā serverī:
 - 4.1. Palaidiet Laravel lokālo serveri: php artisan serve;
 - 4.2. Palaidiet Laravel reverb reāllaika paziņojumiem: php artisan reverb;
 - 4.3. Palaidiet Laravel Queue: php artisan queue:work;
 - 4.4. Palaidiet Next js: npm run dev;
5. Lietotāju pieslēgšanas dati:
 - 5.1. Administratorā profilam var piekļūt ar e-pastu test@example.com un paroli !
Password123;

5.2. Lietotāja profilam var piekļūt ar e-pastu test2@example.com un paroli !
Password123;

6.3. Programmas apraksts

Navigācijas josla

Lapas augšpusē vienmēr ir redzama navigācijas josla. Tās kreisajā pusē atrodas lietojumprogrammas logotips, kas kalpo arī kā saite uz sākumlapu. Blakus logotipam izvietoti navigācijas saites: Pasākumi, Kartes skats un Cilvēki. Ja lietotājs ir pieteicies ar administratora lomu, parādās papildu saite Admin Lietotāji. Labajā pusē pieteiktiem lietotājiem redzamas šādas kontroles: poga Izveidot pasākumu, tiešsaistes draugu logs, paziņojumu zvaniņa ikona, lietotāja vārda nolaižamā izvēlne - ar saitēm uz Profilu), Iestatījumiem un Iziet.



6.1.att. Navigācijas josla

Nepieteiktiem apmeklētājiem navigācijas joslas labajā pusē tiek rādītas saites Pieslēgties un Reģistrēties.

Autentifikācija

Atverot tīmekļa lietotni pārlūkprogrammā, nepieteiktam lietotājam tiek piedāvāts pāriet uz pieslēgšanās lapu, izmantojot saiti navigācijā.

6.2.att. Autentifikācija

Pēc veiksmīgas autentifikācijas lietotājs tiek novirzīts uz sākumlapu. Ja ievadīti nepareizi dati, zem attiecīgajiem laukiem parādās kļūdas paziņojumi.

Reģistrācija

Ja lietotājam vēl nav konta, viņš var doties uz reģistrācijas lapu, izmantojot saiti Registreties navigācijā.



Vārds

E-pasts

Parole

Atkārtot paroli

[Jau esat reģistrējies?](#) [Reģistrēties](#)

6.3.att. Reģistrācija

Aizmirstas paroles atjaunošana

Nospiežot saiti "Aizmirsāt paroli?" pieslēgšanās formā, lietotājs nonāk paroles atjaunošanas lapā, kur var ievadīt savu e-pasta adresi un saņemt atjaunošanas e-pastu.



Aizmirsāt paroli? Nekādu problēmu. Ievadiet savu e-pasta adresi, un mēs nosūtīsim paroles atjaunošanas saiti, kas ļaus izvēlēties jaunu paroli.

E-pasts

Nosūtīt paroles atjaunošanas saiti

6.4.att. Aizmirstas paroles atjaunošana

Pasākumu saraksts

Navigācijā uzklikšķinot uz saites Pasākumi, lietotājs nonāk pasākumu saraksta lapā. Lapas augšdaļā atrodas meklēšanas un filtrēšanas panelis, kuram seko pasākumi.

Meklēt pasākumus

Kategorijas

Meklēt Notīt

☐ Tikai draugi ☐ Tikai tie, kam sekoju

Datuma periods Kārtošana: Gaidāmie pasākumi (Augoši)

Aktuālākie gaidāmie pasākumi

Rāda 11 no 11 pasākumiem



piektd., 26. jūn. 21:10

Rīta skrējiens gar Daugavu

Vanšu tilts, Vecrīga, Latgales apkaime, Rīga, LV-1048, Latvija

Bezmaksas



trešd., 08. jūl. 21:12

Fotogrāfijas meistarklase: Vakara gaisma Vecrīgā

17, Jēzusbaznīcas iela, Lastādija, Latgales apkaime, Rīga, LV-1050, Latvija

Bezmaksas



svētd., 21. jūn. 15:00

Jāņu ielīgošanas vakars Mežaparkā

Dārza iela, Dzirciems, Imantas apkaime, Rīga, LV-1083, Latvija

Bezmaksas

6.5.att. Aizmirstas paroles atjaunošana

Meklēšanas un filtrēšanas panelis satur šādus elementus (skat. 6.5. Att.):

- meklēšanas lauks - teksta ievade pēc pasākuma nosaukuma vai atslēgvārda;
- kategoriju filtrs - izvēle no pieejamajām kategorijām (var izvēlēties vairākas);
- kārtotāšanas opcijas - kārtotāšana pēc dažādiem kritērijiem augošā vai dilstošā secībā;
- poga Meklēt - lai piemērotu izvēlētos filtrus;
- poga Notīrīt - lai notīrītu visus filtrus.

Pieejami tikai pieteiktiem lietotājiem:

- Tikai draugi - rādīt tikai draugu pasākumus;
- Tikai tie, kam sekoju - rādīt tikai sekoto lietotāju pasākumus;

Pasākumu saraksts

Ja nav aktīvu filtru, lapas virsraksts rāda "Aktuālākie gaidāmie pasākumi".

Ja nevienu pasākumu nevar atrast pēc norādītajiem filtriem, tiek parādīts paziņojums:

"Neviens pasākums neatbilda meklēšanai. Mēģiniet citu vietu, kategoriju vai sociālo filtru."

Ja ir vairāk pasākumu nekā redzami, lapas apakšdaļā parādās poga Ielādēt vēl, kura ielādē nākamo pasākumu lapu bez lapas pārlādēšanas.

Pasākuma detaļas lapa

Uzklīkšķinot uz pasākuma kartītes, lietotājs nonāk pasākuma detaļu lapā. Lapa ir sadalīta divās kolonnās. (skat. 6.5.att. un 6.6.att.)



Jāņu ielīgošanas vakars Mežaparkā

Publiskais pasākums

IzklaideLabsajūta un meditācija

0 interesējasEs piedalosDalītiesPievienot kalendāram...

Datums un vieta

2026-06-21 15:00:00

Dārza iela, Dzirciems, Imantas apkaimē, Rīga, LV-1083, Latvija



Par pasākumu

€ Cena

€0.00

Rīkotājs

Test User
test@example.com

Cilvēki, kas piedalās

0 lietotāji atzīmēja, ka piedalās

Uzaicināt draugus

6.5.att. Pasākuma detaļas lapa n1

Par pasākumu

Aicinām uz kopīgu ielīgošanu ar folkloras kopu, ugunscura dziesmām un meistarklasi vainagu pīšanā. Pasākuma laikā būs pieejami vietējo ražotāju gardumi, bezalkoholiskie dzērieni un bērnu radošais stūritis. Līdzī ieteicams paņemt peldu sēdēšanai zālīnā.

Komentāri

Uzraksti komentāru...

0/500

Publicēt

Test User pirms 16 sekundēm (redīgēts)

Nevaru sagaidīt šo pasākumu!!!!

RedīgētDzēst

6.6.att. Pasākuma detaļas lapa n2

Kreisā kolonna:

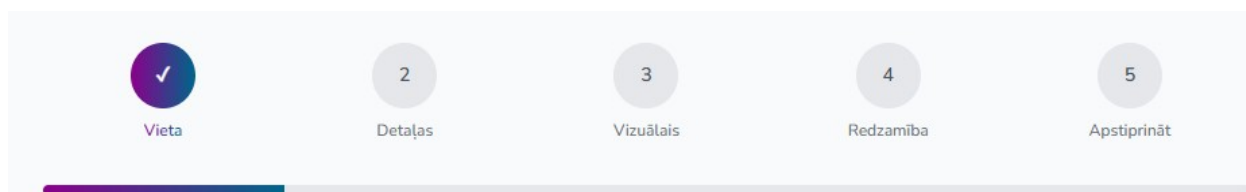
- fona attēls - plaša pasākuma attēla josla lapas augšdaļā;
- pasākuma nosaukums - liels teksts ar redzamību veidu (piemēram, "publisks pasākums", "tikai draugiem", "privāts pasākums");
- kategoriju etiķetes - krāsainas apaļas zīmītes ar pasākuma kategorijām;
- darbību pogas (redzamas pieteiktiem lietotājiem):
- zvaigzne - poga intereses atzīmēšanai (ar to lietotāju skaitu, kuri atzīmējuši interesi);
- piedalos - poga dalības apstiprināšanai;
- dalīties - pasākuma kopīgošanas poga;
- pievienot kalendāram - poga, lai pievienotu pasākumu kalendāram (.ics fails);
- trīs punkti (redzams tikai pasākuma īpašniekam vai administratoram) - izvēlne ar opcijām rediģēt un dzēst.
- datums un vieta - pasākuma sākuma datums un adrese, kā arī interaktīva leaflet karte ar pasākuma atrašanās vietu (skat. att.);
- par pasākumu - pasākuma apraksts markdown formātā;
- komentāru sadaļa - komentāru pievienošana un esošo komentāru apskate.

Labā kolonna:

- cena - pasākuma biļetes cena vai "bezmaksas";
- rīkotājs - pasākuma organizatora vizītkarte ar pogu sekošanai/draudzībai;
- dalībnieku panelis - to lietotāju saraksts, kuri apstiprinājuši dalību;
- dalības pieprasījumu poga - redzama tikai pasākuma īpašniekam, lai pārskatītu ienākošos pieprasījumus.

Pasākuma izveides forma

Nospiežot pogu Izveidot pasākumu navigācijā, autorizēts lietotājs tiek novirzīts uz pasākuma izveides lapu. Lapa ir sadalīta 5 soļos, ko attēlo progressa josla lapas augšdaļā. (skat. 6.7. att.)



6.7.att. Progressa josla

1. solis - Atrašanās vieta (skat. 6.8. att.)

- adrese - adreses meklēšanas lauks ar automātisko pabeigšanu;
- koordinātas - paplašināms bloks koordinātu manuālai ievadei (Platums, Garums);
- interaktīva karte - lietotājs var klikšķināt uz kartes, lai izvēlētos atrašanās vietu; adrese tiek atjaunināta automātiski.

Izvēlieties pasākuma vietu

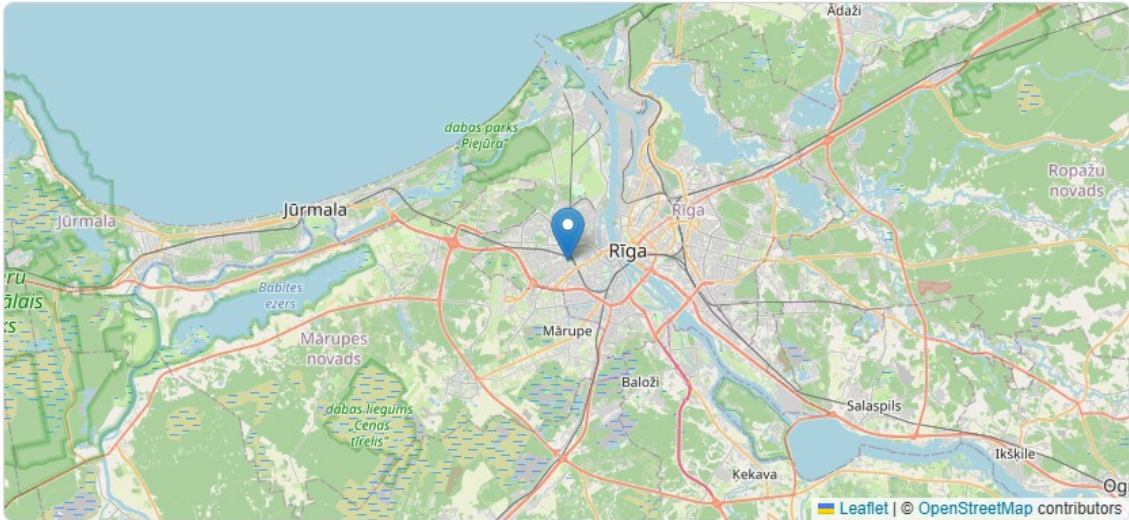
Izvēlieties vietu kartē vai meklējiet adresi Latvijā

Adrese *

GUNSnLASERS izklaides parks, Šampētera iela, Šampēteris, Imantas apkaimē, Rīga, LV-1046, Latvija

Koordinātas

Kartes priekšskats (spiediet, lai izvēlētos vietu)



Izmantojiet adreses meklēšanas lauku augstāk vai klikšķiniet kartē, lai iestatītu precīzu vietu

[← Iepriekšējais](#)[Nākamais →](#)

6.8.att. Vieta

Pēc vietas izvēles jānospiež poga Nākamais, lai pārietu uz nākamo soli.

Cena - biļetes cena (nav obligāta; tukša nozīmē bezmaksas).

6.9.att. Detajlas

3. solis - Vizuālie elementi (skat. 6.10. att.)

Fona attēls - fona attēla augšupielāde no ierīces;

Kategorijas - vienas vai vairāku kategoriju izvēle no pieejamā saraksta.

Vizuālie elementi un kategorijas

Pievienojiet fona attēlu un izvēlieties kategorijas pasākumam

Fona attēls (neobligāti)





Klikšķiniet, lai augšupielādētu, vai ievielciat failu

PNG, JPG, JPEG, SVG vai WebP (maks. 2MB)

Kategorijas (neobligāti) - izvēlieties līdz 5

Muzika un koncerti ✓	Sports	Tehnoloģijas
Bizness un tīklošanās	Māksla un dizains	Ēdiens un dzērieni
Ceļošana un piedzīvojumi	Veselība un fitness	Spēles
Izglītība un mācības	Labdarība un sabiedriskas iniciatīvas	Izklaide ✓

Izvēlētās kategorijas:

Izklaide Ģimenēm draudzīgi Muzika un koncerti

← Iepriekšējais Nākamais →

6.10.att. Vizuāls

4. solis - Redzamība (skat. 6.11. att.)

Lietotājs izvēlas vienu no trim redzamības opcijām:

- Publiski - pasākums ir redzams visiem un tiek rādīts publiskajā meklēšanā un kartē;
- Tikai draugiem - pasākums ir redzams tikai draugiem, publiski netiek rādīts;
- Privāts - pasākums ir pieejams tikai ar saiti.

Pasākuma redzamība

Kontrolējiet, kas var redzēt un atrast jūsu pasākumu

☒ **Publiski**
Ikviens var redzēt šo pasākumu

☐ **Tikai draugiem**
Redz tikai jūsu draugi

☐ **Privāts**
Pieejams tikai ar saiti

Kā tas darbojas:

- ✓ Ikviens var atrast pasākumu meklēšanā
- ✓ Pasākums parādīsies publiskajā kartē

Piezīme: Redzamības iestatījumus varat mainīt jebkurā laikā pēc pasākuma izveides.

← Iepriekšējais

Nākamais →

6.11.att. Redzamība

5. solis - Apstiprinājums (skat. 6.12. att.)

Pēdējais solis parāda visu ievadīto informāciju kopsavilkumā: nosaukumu, aprakstu, datumu, adresi, cenu, kategorijas un fona attēla priekšskatījumu.

Pārskatiet pasākumu

Pirms publicēšanas pārliedzinieties, ka viss ir pareizi



Jāņu ielīgošanas vakars Mežaparkā

GUNSnLASERS izklaides parks, Šampētera iela, Šampēteris, Imantas apkaime, Rīga, LV-1046, Latvija

SĀKUMA DATUMS Sunday, 21 June 2026 at 18:25	BEIGU DATUMS Monday, 22 June 2026 at 00:00	CENA Bezmaksas	REDZAMĪBA Publiski
---	--	-------------------	-----------------------

Par pasākumu

Aicinām uz kopīgu ielīgošanu ar folkloras kopu, ugunscura dziesmām un meistarklasi vainagu pīšanā. Pasākuma laikā būs pieejami vietējo ražotāju gardumi, bezalkoholiskie dzērieni un bērnu radošais stūrītis. Līdzī ieteicams paņemt pledu sēdēšanai zālienā.

Kategorijas

Izklaide Ģimenēm draudzīgi Muzika un koncerti

Koordinātas: 56.9427, 24.0429

6.12.att. Apstiprinājums

Zem kopsavilkuma atrodas Uzaicini draugus - draugu izvēles panelis, kurā var atzīmēt draugus, kas saņems paziņojumu uzreiz pēc pasākuma publicēšanas. (skat. 6.13. att.)

Uzaicini draugus

Izvēlies draugus, kuri saņems paziņojumu uzreiz pēc publicēšanas.

Meklēt draugus pēc vārda vai e-pasta

 **Nikita**
no.accenture.bootcamp@gmail.com

Izvēlēts

Izvēlēti: 1

← Iepriekšējais

Publicēt pasākumu

6.13.att. Uzaicini draugus

Lai publicētu pasākumu, jānospiež poga Publicēt pasākumu. Pēc veiksmīgas publicēšanas lietotājs tiek novirzīts uz sākumlapu. (skat. 6.13. att.)

Ja lapas atstāšanas brīdī veidlapa satur nepublicētas izmaiņas, pārlūkprogramma parāda brīdinājumu par nesaglabātajiem datiem.

Pasākuma rediģēšanas lapa

Uzklikšķinot uz Rediģēt pasākuma īpašnieka izvēlnē, tiek atvērta pasākuma rediģēšanas forma (skat.14-17 att.), kuras struktūra ir identiska pasākuma izveides formai - ar tiem pašiem 5 soļiem (atrašanās vieta, detaļas, vizuālie elementi, redzamība, izņemot apstiprinājumu).

← Atpakaļ uz pasākumu **Rediģēt pasākumu**

Izvēlieties pasākuma vietu

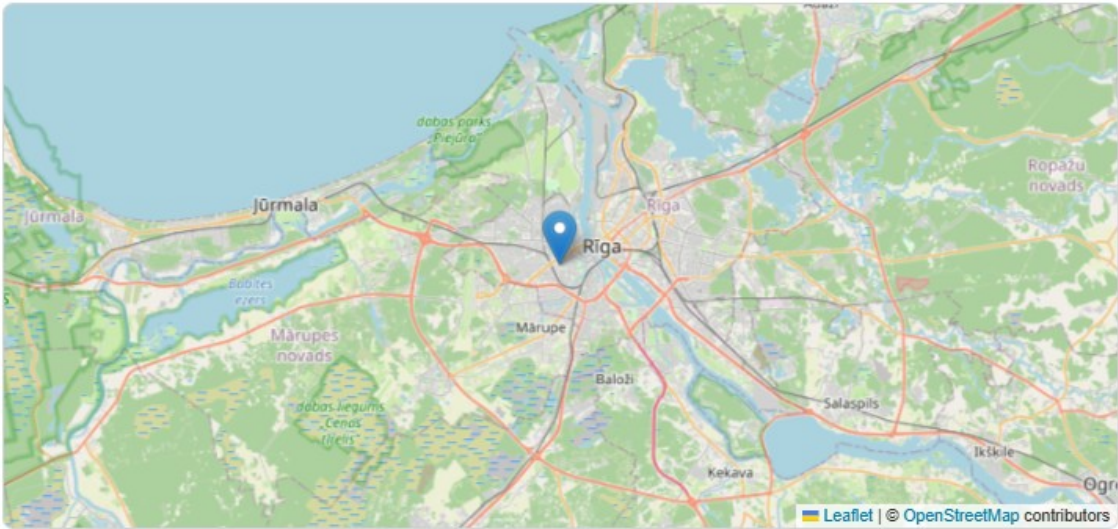
Izvēlieties vietu kartē vai meklējiet adresi Latvijā

Adrese *

39, Ernestīnes iela, Āgenskalna priedes, Āgenskalns, Āgenskalna apkaime, Rīga, LV-1083, Latvija

Koordinātas

Kartes priekšskats (spiediet, lai izvēlētos vietu)



Izmantojiet adreses meklēšanas lauku augstāk vai klikšķiniet kartē, lai iestatītu precīzu vietu

6.14.att. Vieta

Pasākuma detaļas (skat.6.15.att.)

Pasākuma detaļas

Pievienojiet būtiskāko informāciju par savu pasākumu

Nosaukums *

Vasaras brīvdabas kino vakars

29/255 rakstzīmes

Apraksts

B *I* U ~~ABC~~

Brīvdabas kino pasākums, kurā apmeklētājiem ir iespēja baudīt populāras filmas uz lielā ekrāna zem klajas debess. Pasākuma laikā tiek nodrošinātas sēdvietas, uzkodas un dzērieni, kā arī patīkama atmosfēra kopā ar draugiem un ģimeni. Pasākums ir piemērots dažāda vecuma apmeklētājiem un notiek vakara stundās.

...

Izmantojiet rīku joslu virsrakstiem, treknrakstam, slīprakstam, sarakstiem, saitēm un citam.

Sākuma datums un laiks *

Beigu datums un laiks

17-Apr-2026 16:18

dd-----yyyy --:--

Cena (neobligāti) - atstājiet tukšu bezmaksas pasākumam

€ 10.00

6.15.att. Detaļas

Vizuālie elementi un kategorijas(skat.6.16.att.)

Vizuālie elementi un kategorijas

Pievienojiet fona attēlu un izvēlieties kategorijas pasākumam

Fona attēls (neobligāti)



Klikšķiniet, lai augšupielādētu, vai ievielciet failu
PNG, JPG, JPEG, SVG vai WebP (maks. 2MB)

Kategorijas (neobligāti) - izvēlieties līdz 5

Music & Concerts ✓	Sports	Technology
Business & Networking	Art & Design	Food & Drink
Travel & Adventure	Health & Fitness	Gaming
Education & Learning	Charity & Causes	Entertainment ✓

Izvēlētās kategorijas:

Music & Concerts Entertainment Film & Theater

6.16.att. Vizuālie elementi

Pasākuma redzamība (skat.6.17.att.)

Pasākuma redzamība

Kontrolējiet, kas var redzēt un atrast jūsu pasākumu

☒ **Publisks**
Ikviens var redzēt šo pasākumu

☐ **Tikai draugiem**
Redz tikai jūsu draugi

☐ **Privāts**
Pieejams tikai ar saiti

Kā tas darbojas:

- ✓ Ikviens var atrast pasākumu meklēšanā
- ✓ Pasākums parādīsies publiskajā kartē

Piezīme: Redzamības iestatījumus varat mainīt jebkurā laikā pēc pasākuma izveides.

Atcelt

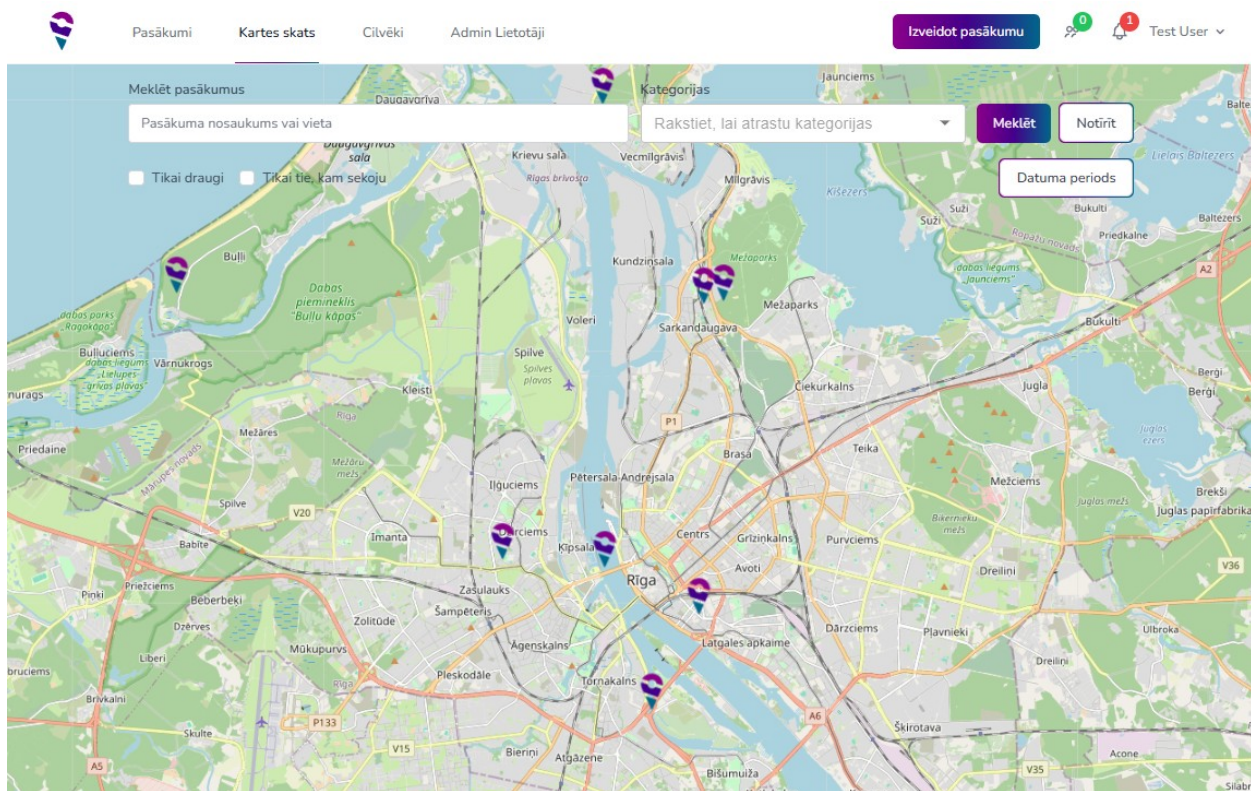
Saglabāt izmaiņas

6.17.att. Redzamība

Kartes skats

Navigācijā uzklikšķinot uz Kartes skats, tiek atvērta interaktīva kartes lapa (skat. 6.18.att.).

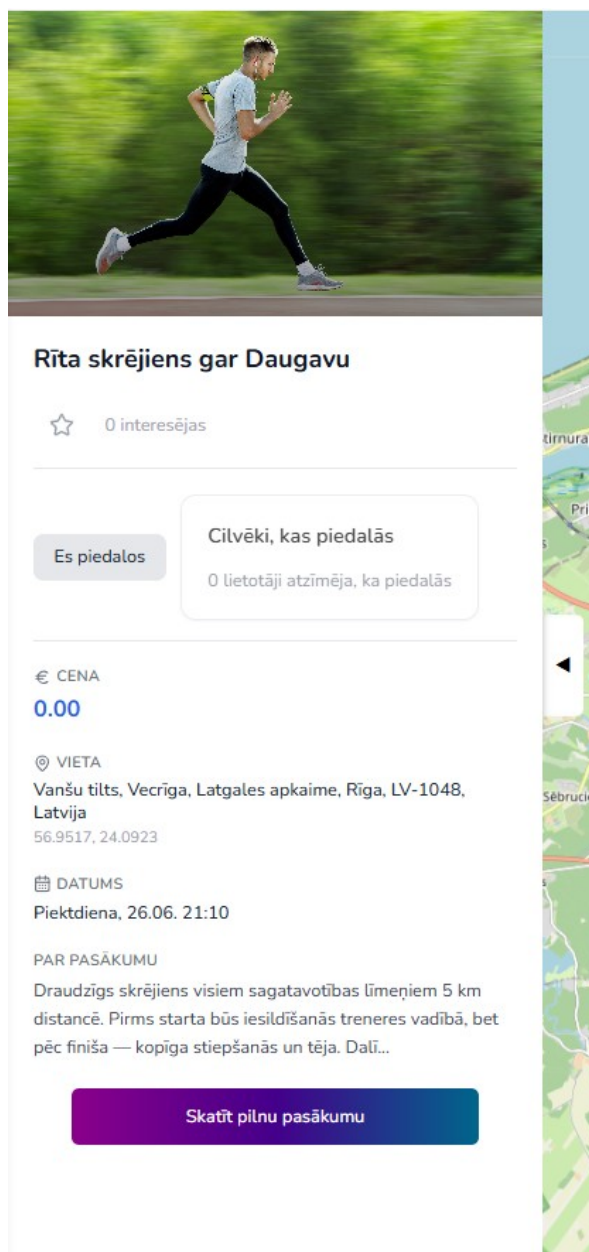
Karte aizpilda visu ekrānu un rāda visus publiski pieejamos pasākumus kā atzīmes kartē.



6.18.att. Kartes skats

Lapas augšdaļā virs kartes atrodas meklēšanas un filtrēšanas panelis (tāds pats kā pasākumu saraksta lapā), kas ļauj filtrēt kartē redzamos pasākumus.

Uzklikšķinot uz atzīmes kartē, lapas kreisajā pusē tiek atvērts pasākuma priekšskatījums sānu panelis ar īsu pasākuma priekšskatījumu un saiti uz pilno pasākuma lapu. (skat. 6.19.att.)



6.19.att. pasākuma priekšskatījums

Ja neviens pasākums neatbilst filtriem, lapas apakšdaļā parādās paziņojums: "Neviens pasākums neatbilda šiem filtriem."

Lietotāju meklēšana (Cilvēki)

Navigācijā uzklikšķinot uz Cilvēki, tiek atvērta lietotāju meklēšanas lapa (skat. 6.20.att.).

Lietotāju meklēšana

Atrodi pasākumu organizatorus vai savus draugus.

 Demo User 01 demo.user01@example.com	 Demo User 10 demo.user10@example.com	 Demo User 11 demo.user11@example.com
 Demo User 12 demo.user12@example.com	 Demo User 13 demo.user13@example.com	 Demo User 14 demo.user14@example.com
 Demo User 15 demo.user15@example.com	 Demo User 16 demo.user16@example.com	 Demo User 17 demo.user17@example.com
 Demo User 18 demo.user18@example.com	 Demo User 19 demo.user19@example.com	

IepriekšējāLapa 1 no 1Nākamā

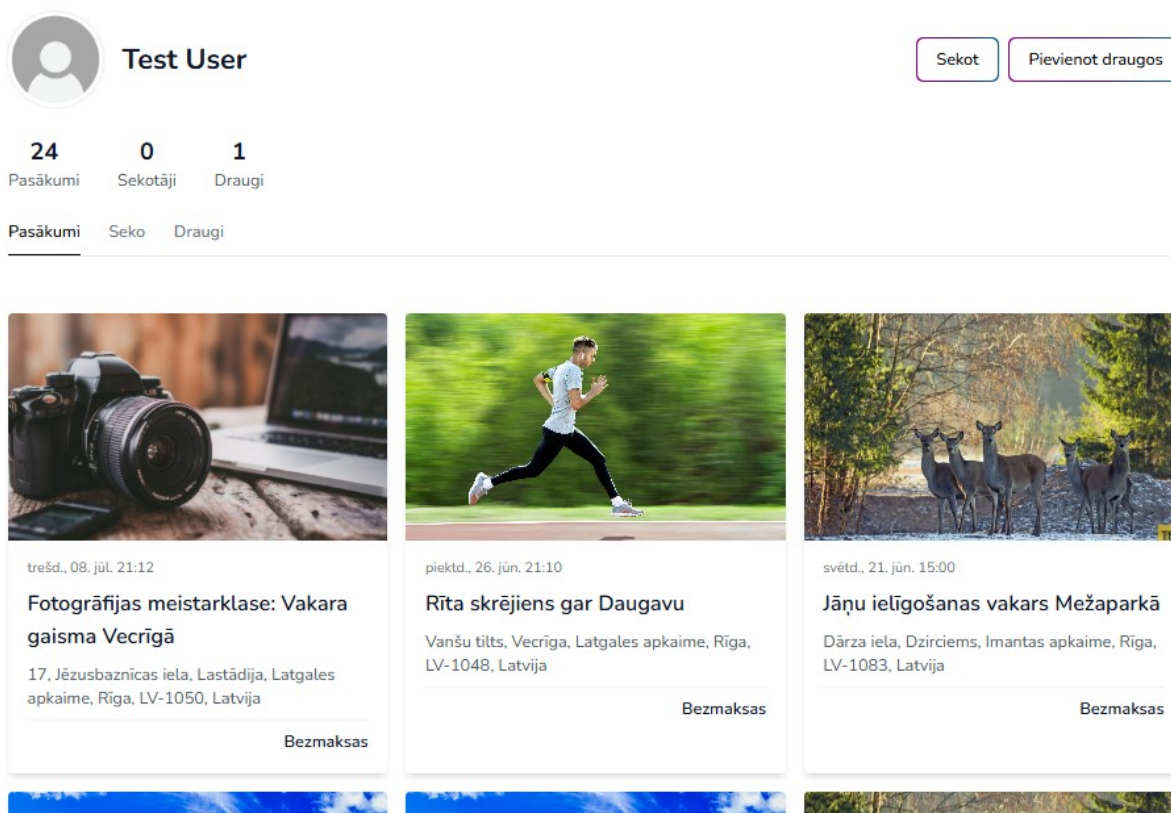
6.20.att. pasākuma priekšskatījums

Atrastie lietotāji tiek attēloti lietotāju kartītēs trīs kolonnās, kur redzami lietotāja vārds, avatārs. Lapas apakšdaļā atrodas lapošanas kontroles navigācijai starp rezultātu lapām.

Ja neviens lietotājs netika atrasts, tiek parādīts paziņojums: "Lietotāji netika atrasti."

Lietotāja profila lapa

Uzklīkšķinot uz kāda lietotāja kartītes vai uz sava vārda navigācijā, tiek atvērta profila lapa (skat. 6.20 att.).



6.20.att. pasākuma priekšskatījums

Darbību pogas (redzamas tikai citiem pieteiktiem lietotājiem):

- Sekot / Seko- poga sekošanai vai sekošanas atcelšanai;
- Pievienot draugos/Pieprasījums nosūtīts/Izņemt no draugiem;

Zem profila galvenes redzama statistikas rinda ar trim skaitītājiem: Pasākumu skaits, Sekotāju skaits, Draugu skaits.

Profila lapa ir sadalīta cilnēs:

- Pasākumi - lietotāja publicētie pasākumi;
- Seko- lietotāji, kuriem sekots;
- Draugi - draugu saraksts;

Iestatījumu lapa

Navigācijas izvēloties Iestatījumi, tiek atvērta iestatījumu lapa. Lapa ir sadalīta divās kolonnās - kreisajā atrodas Sānu josla, bet labajā - aktīvās sadaļas saturs.

Sadaļā Konta informācija (skat. 6.20.att.) lietotājam tiek attēlota rediģējama forma ar laukiem Vārds un E-pasts. Lietotājs var atjaunināt šos datus un saglabāt izmaiņas, nospiežot pogu Saglabāt izmaiņas.

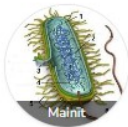
Iestatījumi

Konts

Paziņojumi

Konts

Pārvaldiet sava konta informāciju.



Noklikšķiniet uz attēla, lai to nomainītu.

PAMATINFORMĀCIJA

Vārds

Test User

E-PASTS

test@example.com

PAROLES MAIŅA

JAUNĀ PAROLE

APSTIPRINIET JAUNO PAROLI

Saglabāt izmaiņas

6.20.att. pasākuma priekšskatījums

Sadaļā Paziņojumi (skat. 6.21.att.) lietotājs var pārvaldīt paziņojumu preferences - kāda veida paziņojumus saņemt (piemēram, par pasākumiem, draugu pieprasījumiem u.c.).

Iestatījumi

Konts

Paziņojumi

Paziņojumi

Izvēlieties, kur saņemt katra tipa paziņojumus.

Saņemts draudzības pieprasījums

☒ Lietotne ☐ E-pasts

Draudzības pieprasījums apstiprināts

☒ Lietotne ☐ E-pasts

Saņemts komentārs

☒ Lietotne ☐ E-pasts

Saņemts dalības pieprasījums pasākumam

☒ Lietotne ☐ E-pasts

Sekotais lietotājs izveidoja pasākumu

☒ Lietotne ☐ E-pasts

6.21.att. Paziņojumi

Administratorsa sadaļa - Lietotāju pārvaldīšana

Administratoriem navigācijā tiek rādīta papildu saite Admin Lietotāji, kas atver lietotāju pārvaldīšanas lapu (skat. 6.21.att.). Šī lapa ir pieejama tikai lietotājiem ar administratora lomu.

Lietotāju pārvaldīšana



Demo User 02
demo.user02@example.com

Izvēlēts



Demo User 03
demo.user03@example.com

Izvēlēties



Demo User 04
demo.user04@example.com

Izvēlēties



Demo User 05
demo.user05@example.com

Izvēlēties



Demo User 06
demo.user06@example.com

Izvēlēties



Demo User 07
demo.user07@example.com

Izvēlēties

Lietotāja dati

Vārds

E-pasts

Loma

Lietotājs

▼

Saglabāt

Dzēst lietotāju

Iepriekšējā

Lapa 1 no 4

Nākamā

6.21.att. Paziņojumi

Labajā pusē atrodas panelis ar virsrakstu Lietotāja dati, kurā redzami un rediģējami šādi lauki:

- Vārds - teksta ievades lauks;
- E-pasts - e-pasta ievades lauks;
- Loma - nolaižamā izvēlne ar opcijām "Lietotājs" un "Administrators" ;

Zem laukiem atrodas divas pogas:

- Saglabāt - lai saglabātu izmaiņas (pēc saglabāšanas dati tiek atjaunināti arī sarakstā);
- Dzēst lietotāju - lai dzēstu atlasīto lietotāju.
- Lietotāja dzēšana

6.4. Programmatūras lietojamības testēšana

6.4.1. tabula

Lietotāju pieteikšanās

Lietotāja loma:	Viesis	
Prasības identifikators:	PS1	
Prasības apraksts		
Jānodrošina, lai lietotājs var pieteikties savā kontā		
Ievaddati/situācijas apraksts	Sagaidāmais rezultāts	Statuss
1. Lietotājs ievada pareizu e-pastu un paroli	Lietotājs veiksmīgi pieslēdzas sistēmai	pareizi
2. Lietotājs ievada nepareizu paroli	Sistēma informē, ka parole ir nepareiza	pareizi
3. Lietotājs atstāj e-pata lauku tukšu	Sistēma informē, ka e-pasta adrese ir obligāti aizpildāms lauks	pareizi

6.4.2. tabula

Lietotāju kontu izveidošana

Lietotāja loma:	Lietotājs izveido kontu	
Prasības identifikators:	PS 2	
Prasības apraksts		
Jānodrošina, ka jauns lietotājs tiek izveidots		
Ievaddati/situācijas apraksts	Sagaidāmais rezultāts	Statuss
1. Lietotājvārds, e-pasta adrese un parole ir ievadīti pareizi.	Jauns konts tiek veiksmīgi izveidots	pareizi
2. Nav ievadīts e-pasts	Sistēma informē, ka e-pasta adrese ir obligāti aizpildāms lauks	pareizi
3. Lietotājvārds jau eksistē	Sistēma informē, ka lietotājvārds ir aizņemts	pareizi

Jauns pasākums

Lietotāja loma:	Reģistrēts lietotājs	
Prasības identifikators:	PS 3	
Prasības apraksts		
Jānodrošina, ka jauns pasākums tiek izveidots		
Ievaddati/situācijas apraksts	Sagaidāmais rezultāts	Statuss
1. Pasākuma nosaukums ir tukšs	Sistēma informē, ka nosaukums ir obligāti aizpildāms lauks	pareizi
2. Ievadītie koordināti vai vieta ne eksistē	Sistēma informē, ka adrese ir nepareiza	nepareizi
3. Visi obligātie lauki ir aizpildīti pareizi.	Jauns pasākums tiek veiksmīgi izveidots	pareizi

NOBEIGUMS

Kvalifikācijas darba ietvaros tika izstrādāta tīmekļa sistēma pasākumu pārvaldībai un lietotāju savstarpējai mijiedarbībai. Sistēma nodrošina lietotāju reģistrāciju un autorizāciju, profilu pārvaldību, pasākumu meklēšanu un filtrēšanu, kā arī to attēlošanu kartē. Lietotājiem ir iespēja izveidot un rediģēt pasākumus, pievienot komentārus, izteikt reakcijas, sekot citiem lietotājiem un veidot draudzības saites. Tāpat sistēmā ir ieviesta paziņojumu apstrāde un administratora sadaļa lietotāju pārvaldībai.

Sistēmas izstrādei tika izmantota PHP programmēšanas valoda ar Laravel ietvaru servera pusē, kā arī TypeScript un JavaScript ar Next.js un React klienta pusē. Lietotāja saskarne veidota, izmantojot HTML un CSS ar Tailwind CSS, bet datu glabāšanai izmantota relāciju datubāze. Izstrādes procesā tika nodrošināta stabila sistēmas darbība, korekta piekļuves kontrole atbilstoši lietotāju lomām un pārskatāma saskarne, kas ir ērti lietojama gan datoros, gan mobilajās ierīcēs.

Izstrādāto sistēmu iespējams izmantot kā praktisku risinājumu dažādu pasākumu organizēšanai, dalībnieku piesaistei un kopienu aktivitāšu koordinēšanai, piemēram, studentu vai interešu grupu vidē. Nākotnē sistēmu var paplašināt, ieviešot papildu funkcionalitāti, piemēram, mobilo lietotni, personalizētus ieteikumus, plašākas analītikas iespējas, integrāciju ar kalendāriem un maksājumu sistēmām, kā arī uzlabojot drošības un moderēšanas mehānismus.

INFORMĀCIJAS AVOTI

- Laravel dokumentācija - <https://laravel.com/docs/12.x> - (Resurss apskatīts 15.06.2026.)
- Next.js dokumentācija - <https://nextjs.org/docs> - (Resurss apskatīts 30.03.2026.)
- React dokumentācija - <https://react.dev> - (Resurss apskatīts 30.03.2026.)
- Leaflet dokumentācija - <https://leafletjs.com/reference.html> - (Resurss apskatīts 30.06.2026.)
- Capacitor dokumentācija - <https://capacitorjs.com/docs> - (Resurss apskatīts 30.06.2026.)
- Tailwind dokumentācija - <https://tailwindcss.com/docs/installation/using-vite> - (Resurss apskatīts 30.03.2026.)
- Stackoverflow - <https://stackoverflow.com/> - (Resurss apskatīts 30.03.2026.)
- Material UI - <https://mui.com/material-ui/> - (Resurss apskatīts 30.03.2026.)

PIELIKUMI

1. pielikums

Programmas pirmkods

UserController.php

```
<?php
```

```
/**
```

```
 * Šis kontrolieris nodrošina lietotāja profila un sociālo darbību pārvaldību.
```

```
 * Tas satur astoņas galvenās funkcijas:
```

```
 * - update(): Atjaunina lietotāja profila datus.
```

```
 * - follow(): Sāk vai pārtrauc sekošanu citam lietotājam.
```

```
 * - sendFriendRequest(): Nosūta draudzības pieprasījumu.
```

```
 * - respondFriendRequest(): Pieņem vai noraida draudzības pieprasījumu.
```

```
 * - removeFriend(): Noņem draudzību vai atceļ pieprasījumu.
```

```
 * - friends(): Parāda draugu sarakstu ar tiešsaistes statusu.
```

```
 * - search(): Meklē lietotājus.
```

```
 * - updateOnlineStatus(): Atjaunina lietotāja tiešsaistes statusu.
```

```
 */
```

```
namespace App\Http\Controllers;
```

```
use App\Models\Follower;
```

```
use App\Models\Friend;
```

```
use App\Models\FriendshipRequest;
```

```
use App\Models\User;
```

```
use App\Notifications\FriendRequestAcceptedNotification;
```

```
use App\Notifications\FriendRequestReceivedNotification;
```

```
use App\Support\EventHelper;
```

```
use Illuminate\Http\Request;
```

```
use Illuminate\Support\Facades\Auth;
```

```
use Illuminate\Support\Facades\Cache;
```

```
use Illuminate\Support\Facades\Hash;
```

```
use Illuminate\Support\Facades\Storage;
```

```

use Illuminate\Validation\Rule;
use Throwable;

class UserController extends Controller
{
    // How long will online indicator will live in cash
    private const ONLINE_TTL_SECONDS = 90;

    public function update(Request $request) {
        $user = Auth::user();

        $validated = $request->validate([
            'name' => ['required', 'string', 'max:255'],
            'email' => [
                'required',
                'string',
                'email',
                'max:255',
                Rule::unique('users', 'email')->ignore($user->id),
            ],
            'avatar' => ['nullable', 'file', 'image', 'mimes:jpeg,png,jpg,webp,svg', 'max:4096'],
            'password' => ['nullable', 'string', 'min:8', 'confirmed'],
        ]);

        $updated = [];

        if (isset($validated['name'])) {
            $updated['name'] = $validated['name'];
        }

        if (isset($validated['email'])) {
            if ($validated['email'] !== $user->email) {
                $updated['email_verified_at'] = null;
            }
        }
    }
}

```



```

    }
    $updated['email'] = $validated['email'];
}

$oldAvatarPath = null;
if ($request->hasFile('avatar')) {
    $newAvatarPath = Storage::disk('public')->putFile('AvatarImages', $request->file('avatar'));
    if (!$newAvatarPath) {
        return EventHelper::errorResponse('Neizdevās augšupielādēt profila attēlu.', 500);
    }
    $oldAvatarPath = $user->avatar_path;
    $updated['avatar_path'] = $newAvatarPath;
}

if (!empty($validated['password'])) {
    $updated['password'] = Hash::make($validated['password']);
}
if (!empty($updated)) {
    $user->update($updated);
}

if ($oldAvatarPath && $oldAvatarPath !== ($updated['avatar_path'] ?? null)) {
    Storage::disk('public')->delete($oldAvatarPath);
}

return $user->fresh();
}

public function follow(User $targetUser)
{
    try {
        $authUser = Auth::user();

```

```

if ($authUser->id === $targetUser->id) {
    return EventHelper::errorResponse('Jūs nevarat sekot pats sev', 400);
}

$existing = Follower::where([
    'follower_id' => $authUser->id,
    'target_id' => $targetUser->id,
])->first();

if ($existing) {
    $existing->delete();
    $isFollowing = false;
} else {
    Follower::create([
        'follower_id' => $authUser->id,
        'target_id' => $targetUser->id,
    ]);
    $isFollowing = true;
}

return EventHelper::successResponse(
    message: $isFollowing ? 'Jūs veiksmīgi sekojat lietotājam' : 'Jūs veiksmīgi pārtraucāt
sekošanu'
);

} catch (Throwable $e) {
    \Log::error('Follow failed', [
        'error' => $e->getMessage(),
    ]);

    return EventHelper::errorResponse('Radās kļūda', 500);
}
}

```

```

/**
 * Send a friend request to $targetUser.
 */
public function sendFriendRequest(User $targetUser)
{
    try {
        $authUser = Auth::user();

        if ($authUser->id === $targetUser->id) {
            return EventHelper::errorResponse('Jūs nevarat pievienot pats sevi', 400);
        }

        if (Friend::areFriends($authUser->id, $targetUser->id)) {
            return EventHelper::errorResponse('Jūs jau esat draugi', 409);
        }

        if (FriendshipRequest::getBetween($authUser->id, $targetUser->id)) {
            return EventHelper::errorResponse('Draudzības pieprasījums jau pastāv', 409);
        }

        FriendshipRequest::create([
            'user_id' => $authUser->id,
            'receiver_id' => $targetUser->id,
            'status' => 'pending',
        ]);

        $targetUser->notify(new FriendRequestReceivedNotification($authUser));

        return EventHelper::successResponse('Draudzības pieprasījums nosūtīts');
    } catch (Throwable $e) {
        \Log::error('sendFriendRequest failed', ['error' => $e->getMessage()]);
        return EventHelper::errorResponse('Radās kļūda', 500);
    }
}

```

```

    }
}

/**
 * Accept or decline a friend request from $targetUser.
 */
public function respondFriendRequest(User $targetUser, Request $request)
{
    try {
        $request->validate(['action' => 'required|in:accept,decline']);

        $authUser = Auth::user();

        $friendRequest = FriendshipRequest::where('user_id', $targetUser->id)
            ->where('receiver_id', $authUser->id)
            ->where('status', 'pending')
            ->firstOrFail();

        if ($request->action === 'accept') {
            $friendRequest->update(['status' => 'accepted']);
            Friend::create(['user1_id' => $targetUser->id, 'user2_id' => $authUser->id]);

            $targetUser->notify(new FriendRequestAcceptedNotification($authUser));

            return EventHelper::successResponse('Draudzības pieprasījums pieņemts');
        } else {
            $friendRequest->update(['status' => 'declined']);
            return EventHelper::successResponse('Draudzības pieprasījums noraidīts');
        }
    } catch (Throwable $e) {
        \Log::error('respondFriendRequest failed', ['error' => $e->getMessage()]);
        return EventHelper::errorResponse('Radās kļūda', 500);
    }
}

```

```

}

/**
 * Cancel an outgoing friend request OR remove an existing friendship.
 */
public function removeFriend(User $targetUser)
{
    try {
        $authUser = Auth::user();

        // Cancel outgoing request
        FriendshipRequest::where('user_id', $authUser->id)
            ->where('receiver_id', $targetUser->id)
            ->delete();

        // Remove friendship
        Friend::removeFriendship($authUser->id, $targetUser->id);

        return EventHelper::successResponse('Draudzība noņemta');
    } catch (Throwable $e) {
        \Log::error('removeFriend failed', ['error' => $e->getMessage()]);
        return EventHelper::errorResponse('Radās kļūda', 500);
    }
}

/**
 * Return authenticated user friends with computed online state.
 */
public function friends(Request $request)
{
    $authUser = Auth::user();

    $search = trim((string) $request->query('search', ''));

```

```
$limit = (int) $request->query('limit', 200);
```

```
$limit = max(1, min($limit, 500));
```

```
$query = $authUser->friends()->select(['users.id', 'users.name', 'users.email',  
'users.avatar_path']);
```

```
if ($search !== "") {  
    $query->where(function ($builder) use ($search) {  
        $builder  
            ->where('users.name', 'like', "%{$search}%")  
            ->orWhere('users.email', 'like', "%{$search}%");  
    });  
}
```

```
$friends = $query  
    ->orderBy('users.name')  
    ->limit($limit)  
    ->get();
```

```
$now = now();
```

```
$friendsPayload = $friends->map(function (User $friend) use ($now) {  
    $lastSeenAt = Cache::get($this->onlineCacheKey($friend->id));  
    $isOnline = false;
```

```
    if ($lastSeenAt) {  
        try {  
            $isOnline = $now->diffInSeconds(\Carbon\Carbon::parse($lastSeenAt)) <=  
self::ONLINE_TTL_SECONDS;  
        } catch (\Throwable $exception) {  
            $isOnline = false;  
        }  
    }  
}
```

```

        return [
            'id' => $friend->id,
            'name' => $friend->name,
            'email' => $friend->email,
            'avatar_path' => $friend->avatar_path,
            'is_online' => $isOnline,
        ];
    }) // Sort that online friends are on top, and then by name
    ->sort(function (array $left, array $right) {
        if ($left['is_online'] !== $right['is_online']) {
            return $left['is_online'] ? -1 : 1;
        }

        return strcasecmp($left['name'], $right['name']);
    })
    ->values()
    ->all();

return EventHelper::successResponse(data: $friendsPayload);
}

public function search(Request $request)
{
    $authUser = Auth::user();

    $validated = $request->validate([
        'search' => 'nullable|string|max:255',
        'page' => 'nullable|integer|min:1',
        'per_page' => 'nullable|integer|min:1|max:50',
    ]);

    $search = trim($validated['search'] ?? '');

```

```

$perPage = (int) ($validated['per_page'] ?? 15);

$query = null;

if ($authUser) {
    $query = User::query()
        ->select(['id', 'name', 'email', 'avatar_path'])
        ->where('id', '!=', $authUser->id);
} else
{
    $query = User::query()
        ->select(['id', 'name', 'email', 'avatar_path']);
}

if ($search !== "") {
    $query->where(function ($builder) use ($search) {
        $builder
            ->where('name', 'like', "%{$search}%")
            ->orWhere('email', 'like', "%{$search}%");
    });
}

$users = $query
    ->orderBy('name')
    ->paginate($perPage);

return EventHelper::successResponse(
    data: $users->items(),
    meta: [
        'current_page' => $users->currentPage(),
        'per_page' => $users->perPage(),
        'total' => $users->total(),
        'last_page' => $users->lastPage(),
    ]
);

```



```

    ],
);
}

/**
 * Update authenticated user online status timestamp
 */
public function updateOnlineStatus()
{
    $authUser = Auth::user();
    Cache::put($this->onlineCacheKey($authUser->id), now()->toIso8601String(), now()->addSeconds(self::ONLINE_TTL_SECONDS * 2));

    return EventHelper::successResponse('Tiešsaistes statuss ir atjaunināts');
}

// Helper functions

private function onlineCacheKey(int $userId): string
{
    return "user-online:{$userId}";
}
}

```

```
<?php
```

```
/**
```

```
 * Šis kontrolieris nodrošina lietotāja paziņojumu pārvaldību.
```

```
 * Tas satur piecas galvenās funkcijas:
```

```
 * - index(): Parāda paziņojumu sarakstu ar nelasīto skaitu.
```

```
 * - markAsRead(): Atzīmē vienu paziņojumu kā izlasītu.
```

```
 * - markAllAsRead(): Atzīmē visus paziņojumus kā izlasītus.
```

```
 * - delete(): Dzēš vienu paziņojumu.
```

```
 * - deleteAll(): Dzēš visus lietotāja paziņojumus.
```

```
 */
```

```
namespace App\Http\Controllers;
```

```
use App\Support\EventHelper;
```

```
use App\Support\NotificationMapper;
```

```
use Illuminate\Http\JsonResponse;
```

```
use Illuminate\Http\Request;
```

```
use Illuminate\Notifications\DatabaseNotification;
```

```
class NotificationController extends Controller
```

```
{
```

```
    public function index(Request $request)
```

```
    {
```

```
        $user = $request->user();
```

```
        $perPage = min((int) $request->integer('per_page', 10), 50);
```

```
        $notifications = $user->notifications()->latest()->paginate($perPage);
```

```
        $notifications->setCollection(
```

```
            $notifications->getCollection()->map(
```

```
                fn (DatabaseNotification $notification): array =>
```

```
                NotificationMapper::fromDatabase($notification)
```

```

    )
);

return response()->json([
    'data' => $notifications,
    'meta' => [
        'unread_count' => $user->unreadNotifications()->count(),
    ],
]);
}

public function markAsRead(Request $request, string $notificationId)
{
    $notification = $this->ownedNotification($request, $notificationId);

    if (!$notification) {
        return EventHelper::errorResponse('Paziņojums nav atrasts', 404);
    }

    if (!$notification->read_at) {
        $notification->markAsRead();
    }

    return EventHelper::successResponse();
}

public function markAllAsRead(Request $request)
{
    $request->user()->unreadNotifications()->update([
        'read_at' => now(),
    ]);

    return EventHelper::successResponse();
}

```

```

}

public function delete(Request $request, string $notificationId)
{
    $notification = $this->ownedNotification($request, $notificationId);

    if (!$notification) {
        return EventHelper::errorResponse('Paziņojums nav atrasts', 404);
    }

    $notification->delete();

    return EventHelper::successResponse();
}

public function deleteAll(Request $request)
{
    $request->user()->notifications()->delete();

    return EventHelper::successResponse();
}

// Gets notification by id
private function ownedNotification(Request $request, string $notificationId):
DatabaseNotification
{
    return $request->user()
        ->notifications()
        ->whereKey($notificationId)
        ->first();
}
}

```

```

<?php
/**
 * Šis kontrolieris nodrošina lietotāja paziņojumu pārvaldību.
 * Tas satur piecas galvenās funkcijas:
 * - index(): Parāda paziņojumu sarakstu ar nelasīto skaitu.
 * - markAsRead(): Atzīmē vienu paziņojumu kā izlasītu.
 * - markAllAsRead(): Atzīmē visus paziņojumus kā izlasītus.
 * - delete(): Dzēš vienu paziņojumu.
 * - deleteAll(): Dzēš visus lietotāja paziņojumus.
 */

namespace App\Http\Controllers;

use App\Support\EventHelper;
use App\Support\NotificationMapper;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;
use Illuminate\Notifications\DatabaseNotification;

class NotificationController extends Controller
{
    public function index(Request $request)
    {
        $user = $request->user();

        $perPage = min((int) $request->integer('per_page', 10), 50);
        $notifications = $user->notifications()->latest()->paginate($perPage);

        $notifications->setCollection(
            $notifications->getCollection()->map(

```

```

        fn (DatabaseNotification $notification): array =>
NotificationMapper::fromDatabase($notification)
    )
);

return response()->json([
    'data' => $notifications,
    'meta' => [
        'unread_count' => $user->unreadNotifications()->count(),
    ],
]);
}

public function markAsRead(Request $request, string $notificationId)
{
    $notification = $this->ownedNotification($request, $notificationId);

    if (!$notification) {
        return EventHelper::errorResponse('Paziņojums nav atrasts', 404);
    }

    if (!$notification->read_at) {
        $notification->markAsRead();
    }

    return EventHelper::successResponse();
}

public function markAllAsRead(Request $request)
{
    $request->user()->unreadNotifications()->update([
        'read_at' => now(),
    ]);
}

```

```

        return EventHelper::successResponse();
    }

    public function delete(Request $request, string $notificationId)
    {
        $notification = $this->ownedNotification($request, $notificationId);

        if (!$notification) {
            return EventHelper::errorResponse('Paziņojums nav atrasts', 404);
        }

        $notification->delete();

        return EventHelper::successResponse();
    }

    public function deleteAll(Request $request)
    {
        $request->user()->notifications()->delete();

        return EventHelper::successResponse();
    }
    // Gets notification by id
    private function ownedNotification(Request $request, string $notificationId):
    DatabaseNotification
    {
        return $request->user()
            ->notifications()
            ->whereKey($notificationId)
            ->first();
    }
}

```

```
<?php
```

```
/**
```

```
 * Šis kontrolieris nodrošina jauna lietotāja reģistrāciju.
```

```
 * Tas satur galveno funkciju:
```

```
 * - store(): Reģistrē jaunu lietotāju un pieslēdz to sistēmai.
```

```
 */
```

```
namespace App\Http\Controllers\Auth;
```

```
use App\Http\Controllers\Controller;
```

```
use App\Models\User;
```

```
use Exception;
```

```
use Illuminate\Auth\Events\Registered;
```

```
use Illuminate\Http\Request;
```

```
use Illuminate\Http\Response;
```

```
use Illuminate\Support\Facades\Auth;
```

```
use Illuminate\Support\Facades\Hash;
```

```
use Illuminate\Support\Facades\Log;
```

```
use Illuminate\Validation\Rules;
```

```
class RegisteredUserController extends Controller
```

```
{
```

```
 /**
```

```
 * Handle an incoming registration request.
```

```
 *
```

```
 * @throws \Illuminate\Validation\ValidationException
```

```
 */
```

```
public function store(Request $request)
```

```
{
```

```
    $request->validate([
```

```
        'name' => ['required', 'string', 'max:255', 'unique:users,name'],
```

```
        'email' => ['required', 'string', 'lowercase', 'email', 'max:255', 'unique:'.User::class],
```



```

        'password' => ['required', 'confirmed', Rules\Password::defaults()],
    ]);

    $user = User::create([
        'name' => $request->name,
        'email' => $request->email,
        'password' => Hash::make($request->string('password')),
    ]);

    event(new Registered($user));

    Auth::login($user);
    $isNativeClient = $request->header('X-Client-Platform') === 'native';

    if (!$isNativeClient) {
        try {
            Log::info('Session ID:', [$request->session()->getId()]);
        } catch (Exception $e) {
            Log::error("No session stored in headers");
        }

        return response()->json($user);
    }

    // Mobile - create token
    $token = $user->createToken(config('app.name'))->plainTextToken;

    return response()->json([
        'user' => $user,
        'token' => $token,
    ]);
}
}

```

```
<?php
```

```
/**
```

```
 * Šis modelis nodrošina pasākumu datu glabāšanu, saites ar citām entītijām un dalībnieku attiecību iegūšanu.
```

```
 * Tas satur desmit galvenās funkcijas:
```

```
 * - getLatAttribute(): Atgriež pasākuma platumu no saistītās adreses.
```

```
 * - address(): Atgriež ar pasākumu saistīto adresi.
```

```
 * - comments(): Atgriež pasākuma komentārus.
```

```
 * - user(): Atgriež pasākuma autoru.
```

```
 * - interestedUsersCount(): Atgriež interesentu skaitu.
```

```
 * - interestedUsers(): Atgriež interesentu attiecību ierakstus.
```

```
 * - goingUsersCount(): Atgriež apmeklētāju skaitu.
```

```
 * - goingUsers(): Atgriež apmeklētāju attiecību ierakstus.
```

```
 * - categories(): Atgriež pasākumam piešķirtās kategorijas.
```

```
 * - visibility(): Atgriež pasākuma redzamības tipu.
```

```
 */
```

```
namespace App\Models;
```

```
use Illuminate\Database\Eloquent\Factories\HasFactory;
```

```
use Illuminate\Database\Eloquent\Model;
```

```
class Event extends Model
```

```
{
```

```
    use HasFactory;
```

```
    protected $fillable = [
```

```
        'title',
```

```
        'description',
```

```
        'start_date',
```

```
        'end_date',
```

```
        'price',
```

```
'background_image_path',  
'lat',  
'lng',  
'user_id',  
'address_id',  
'event_visibility_id',  
];
```

```
public function getLatAttribute()  
{  
    return $this->address?->lat;  
}
```

```
public function address() {  
    return $this->belongsTo(Address::class);  
}
```

```
public function comments() {  
    return $this->hasMany(Comment::class);  
}
```

```
public function user() {  
    return $this->belongsTo(User::class);  
}
```

```
public function interestedUsersCount()  
{  
    return $this->hasMany(EventInterested::class)->count();  
}
```

```
public function interestedUsers()  
{  
    return $this->hasMany(EventInterested::class);  
}
```

```

    }

    public function goingUsersCount()
    {
        return $this->hasMany(EventGoing::class)->count();
    }

    public function goingUsers()
    {
        return $this->hasMany(EventGoing::class);
    }

    public function categories()
    {
        return $this->belongsToMany(Category::class, 'event_category_mappings', 'event_id',
'category_id');
    }

    public function visibility()
    {
        return $this->belongsTo(EventVisibility::class, 'event_visibility_id');
    }
}

```

```
<?php
```

```
namespace App\Notifications;
```

```
use App\Enums\NotificationType;
```

```
use App\Models\Event;
```

```
use App\Models\User;
```

```
use App\Support\NotificationMapper;
```

```
use Illuminate\Bus\Queueable;
```

```
use Illuminate\Contracts\Queue\ShouldQueue;
```

```
use Illuminate\Notifications\Messages\BroadcastMessage;
```

```
use Illuminate\Notifications\Messages\MailMessage;
```

```
use Illuminate\Notifications\Notification;
```

```
class EventParticipationRequestNotification extends Notification implements ShouldQueue
```

```
{
```

```
    use Queueable;
```

```
    public function __construct(
```

```
        private readonly User $sender,
```

```
        private readonly Event $event,
```

```
    ) {
```

```
    }
```

```
    public function via(object $notifiable): array
```

```
    {
```

```
        if (!$notifiable instanceof User) {
```

```
            return [];
```

```
        }
```

```
        return $notifiable->
```

```
notificationChannelsForType(NotificationType::EventParticipationRequest->value);
```

```
}
```

```
public function toMail(object $notifiable): MailMessage
```

```
{
```

```
    return (new MailMessage)
```

```
        ->subject('Uzaicinājums piedalīties pasākumā')
```

```
        ->greeting('Sveiki!')
```

```
        ->line("{$this->sender->name} uzaicināja jūs pievienoties pasākumam \"{$this->event->title}\".")
```

```
        ->line('Atveriet lietotni, lai apskatītu pasākuma informāciju.');
```

```
}
```

```
public function toBroadcast(object $notifiable): BroadcastMessage
```

```
{
```

```
    return new BroadcastMessage(
```

```
        NotificationMapper::fromBroadcast($this, $this->toArray($notifiable))
```

```
    );
```

```
}
```

```
public function toArray(object $notifiable): array
```

```
{
```

```
    return [
```

```
        'notification_type' => NotificationType::EventParticipationRequest->value,
```

```
        'title' => 'Uzaicinājums piedalīties pasākumā',
```

```
        'body' => "{$this->sender->name} uzaicināja jūs pievienoties pasākumam \"{$this->event->title}\".",
```

```
        'actor' => [
```

```
            'id' => $this->sender->id,
```

```
            'name' => $this->sender->name,
```

```
            'avatar' => $this->sender->avatar_path,
```

```
        ],
```

```
        'resource' => [
```

```
            'type' => 'event',
```

```
        'id' => $this->event->id,  
    ],  
];  
}  
}
```

```
"use client";
```

```
/**
```

```
 * Šī lapa ļauj pasākuma rīkotājam vai administratoram rediģēt esošus pasākumus.
```

```
 * Atbalsta vietas, detaļu, vizuālo elementu un redzamības iestatījumu maiņu ar pilnīgu  
apstiprināšanu.
```

```
*/
```

```
import { getPost, updatePost } from '@utils/post_service';
import type { UpdatePostData, EventResponse } from '@utils/post_service';
import { getCategories } from '@utils/category_service';
import { useEffect, useState, useCallback } from 'react';
import { Category, EventFormData } from '@utils/Types';
import { useRouter } from 'next/navigation';
import Loading from '@components/Loading';
import { useAuth } from '@hooks/auth';
import LocationStage from '@app/(app)/create-event/Stage1Location';
import DetailsStage from '@app/(app)/create-event/Stage2Details';
import VisualsStage from '@app/(app)/create-event/Stage3Visuals';
import VisibilityStage from '@app/(app)/create-event/Stage4Visibility';
import { validateEventData } from '@utils/eventValidation';
import { useUnsavedChanges } from '@hooks/useUnsavedChanges';
```

```
export default function EditEventPage() {
  const [eventId, setEventId] = useState<string | null>(null);
  const [loading, setLoading] = useState(true);
  const [isSaving, setIsSaving] = useState(false);
  const [isDirty, setIsDirty] = useState(false);
  const [pageError, setPageError] = useState<string | null>(null);
  const [saveError, setSaveError] = useState<string | null>(null);
  const [categories, setCategories] = useState<Category[]>([]);
  const [loadingCategories, setLoadingCategories] = useState(true);
```



```

const [formData, setFormData] = useState<EventFormData>({
  title: "",
  description: "",
  address: "",
  latitude: null,
  longitude: null,
  startDate: "",
  endDate: "",
  price: "",
  visibility: 'public',
  categories: [],
  backgroundImage: null,
  inviteeIds: [],
  errors: {},
});

const router = useRouter();
const { user } = useAuth();

useUnsavedChanges(isDirty);

const toDateTimeLocalValue = (value?: string) => {
  if (!value) return "";
  const date = new Date(value);
  if (Number.isNaN(date.getTime())) return "";
  const pad = (num: number) => String(num).padStart(2, '0');
  return `${date.getFullYear()}-${pad(date.getMonth() + 1)}-${pad(date.getDate())}T${
pad(date.getHours())}:${pad(date.getMinutes())}`;
};

// fetch categories
useEffect(() => {
  getCategories()

```

```

        .then(setCategories)
        .catch(console.error)
        .finally(() => setLoadingCategories(false));
}, []);

// fetch event data
useEffect(() => {
    const id = typeof window !== 'undefined'
        ? new URLSearchParams(window.location.search).get('id')
        : null;
    setEventId(id);

    if (!id) {
        setPageError('Pasākuma ID nav nodots.');
```

setLoading(false);

return;

}

// Wait for auth state to resolve before checking ownership.

if (typeof user === 'undefined') {

return;

}

setLoading(true);

setPageError(null);

async function fetchEvent() {

try {

const response: EventResponse = await getPost(id!);

const event = response.data;

const host = response.meta.*host*;

const canManageEvent = Boolean(user && (user.role === 'admin' || (host && user.id

```
=== host.id)));
```

```
if (!canManageEvent) {  
    setPageError('Jums nav tiesību rediģēt šo pasākumu.');
```

```
    setLoading(false);
```

```
    return;
```

```
}
```

```
setFormData({  
    title: event.title ?? "",  
    description: event.description ?? "",  
    address: event.address?.name ?? "",  
    latitude: event.address?.lat ?? null,  
    longitude: event.address?.lng ?? null,  
    startDate: toDateTimeLocalValue(event.start_date),  
    endDate: toDateTimeLocalValue(event.end_date),  
    price: event.price ? String(event.price) : "",  
    visibility: event.visibility ?? 'public',  
    categories: event.categories?.map((category) => category.id) ?? [],  
    backgroundImage: null,  
    inviteeIds: [],  
    errors: {},  
});
```

```
setPageError(null);
```

```
} catch (e: any) {
```

```
    setPageError(e?.response?.data?.message ?? 'Neizdevās ielādēt pasākumu.');
```

```
} finally {
```

```
    setLoading(false);
```

```
}
```

```
}
```

```
fetchEvent();
```

```
}, [user]);
```

```

const validate = useCallback(() : boolean => {
  const newErrors = validateEventData(formData, 'all');
  setFormData((prev) => ({ ...prev, errors: newErrors }));
  return Object.keys(newErrors).length === 0;
}, [formData]);

const updateForm = useCallback((partial: Partial<EventFormData>) => {
  setIsDirty(true);
  setFormData((prev) => ({ ...prev, ...partial }));
}, []);

async function handleSave() {
  if (!eventId || isSaving) return;
  if (!validate()) {
    setSaveError('Lūdzu izlabojiet kļūdas pirms saglabāšanas.');
    return;
  }

  setIsSaving(true);
  setSaveError(null);

  try {
    const payload: UpdatePostData = {
      title: formData.title.trim(),
      description: formData.description.trim(),
      address_name: formData.address.trim(),
      start_date: new Date(formData.startDate).toISOString(),
      lat: Number(formData.latitude),
      lng: Number(formData.longitude),
      visibility: formData.visibility,
    };
    if (formData.endDate) payload.end_date = new Date(formData.endDate).toISOString();
  }
}

```

```

    if (formData.price) payload.price = formData.price;

    const response = await updatePost(eventId, payload);

    if (response.status !== 'ok') {
      setSaveError(response.message ?? 'Neizdevās saglabāt izmaiņas.');
      return;
    }

    setIsDirty(false);
    router.push(`/event?id=${eventId}`);
  } catch (e: any) {
    setSaveError(e?.response?.data?.message ?? 'Neizdevās saglabāt izmaiņas.');
  } finally {
    setIsSaving(false);
  }
}

if (loading) return <Loading />;

if (pageError) {
  return (
    <div className="max-w-3xl mx-auto py-12 px-4">
      <div className="rounded-lg border border-red-200 bg-red-50 p-4 text-red-
800">{pageError}</div>
    </div>
  );
}

return (
  <div className="max-w-4xl mx-auto py-10 px-4">
    { /* Header */ }
    <div className="mb-6 flex items-center gap-3">

```

```

<button
  type="button"
  onClick={() => router.push(`/event?id=${eventId}`)}
  className="text-sm text-gray-500 hover:text-gray-700"
>
  ← Atpakaļ uz pasākumu
</button>

<h1 className="text-2xl font-bold text-gray-900">Rediģēt pasākumu</h1>
</div>

```

```

{saveError && (
  <div className="mb-6 rounded-md border border-red-200 bg-red-50 px-4 py-3 text-sm text-red-800">
    {saveError}
  </div>
)}

```

```

<div className="space-y-8">
  {/* Location */}
  <section className="bg-white rounded-xl border shadow-sm p-6">
    <LocationStage
      address={formData.address}
      latitude={Number(formData.latitude)}
      longitude={Number(formData.longitude)}
      onAddressChange={({address}) => updateForm({ address })}
      onCoordinatesChange={({lat, lng}) => updateForm({ latitude: lat, longitude: lng })}
      error={formData.errors.location ?? formData.errors.address}
    />
  </section>

```

```

  {/* Details */}
  <section className="bg-white rounded-xl border shadow-sm p-6">
    <DetailsStage

```

```

    title={formData.title}
    description={formData.description}
    startDate={formData.startDate}
    endDate={formData.endDate}
    price={formData.price}
    onTitleChange={({title}) => updateForm({ title })}
    onDescriptionChange={({description}) => updateForm({ description })}
    onStartDateChange={({startDate}) => updateForm({ startDate })}
    onEndDateChange={({endDate}) => updateForm({ endDate })}
    onPriceChange={({price}) => updateForm({ price })}
    errors={formData.errors}
  />
</section>

```

```

{/* Visuals & Categories */}
<section className="bg-white rounded-xl border shadow-sm p-6">
  <VisualsStage
    categories={formData.categories}
    categoryList={categories}
    loadingCategories={loadingCategories}
    onBackgroundImageChange={({backgroundImage}) =>
updateForm({ backgroundImage })}
    onCategoriesChange={({cats}) => updateForm({ categories: cats })}
    errors={formData.errors}
  />
</section>

```

```

{/* Visibility */}
<section className="bg-white rounded-xl border shadow-sm p-6">
  <VisibilityStage
    visibility={formData.visibility}
    onVisibilityChange={({visibility}) => updateForm({ visibility })}
  />

```

```

    </section>
  </div>

  { /* Footer actions */ }
  <div className="mt-8 flex justify-end gap-3">
    <button
      type="button"
      onClick={() => router.push(`/event?id=${eventId}`)}
      className="rounded-lg border border-gray-300 px-5 py-2.5 text-sm hover:bg-gray-
50"
    >
      Atcēlīt
    </button>
    <button
      type="button"
      onClick={handleSave}
      disabled={isSaving}
      className="rounded-lg bg-indigo-600 px-5 py-2.5 text-sm font-medium text-white
hover:bg-indigo-700 disabled:opacity-60"
    >
      {isSaving ? 'Saglabāju...' : 'Saglabāt izmaiņas'}
    </button>
  </div>
</div>
);
}

```



```
"use client";
```

```
/**
```

```
 * Šī lapa parāda pasākumu sarakstu ar papildu meklēšanas, filtrēšanas un kārtošanas iespējām.
```

```
 * Atbalsta infīnīto ielādi un ļauj lietotājam redzēt kopējo pasākumu skaitu.
```

```
*/
```

```
import { useContext, useEffect, useState } from 'react'
```

```
import EventSearchAndFilters from '@components/Event/EventSearchAndFilters';
```

```
import { useAuth } from '@hooks/auth';
```

```
import type { EventFilters, EventType } from '@utils/Types';
```

```
import { getPosts, type PaginatedEventsMeta } from '@utils/post_service';
```

```
import Loading from '@components/Loading';
```

```
import EventCard from '@components/Event/EventCard';
```

```
import { SnackbarContext } from '@context/SnackbarContext'
```

```
import { extractErrorMessage, extractValidationErrors, isValidValidationError } from  
'@utils/response_helper'
```

```
const DEFAULT_FILTERS: EventFilters = {
```

```
  search: "",
```

```
  categories: [],
```

```
  friends_only: false,
```

```
  following_only: false,
```

```
  date_from: "",
```

```
  date_to: "",
```

```
  sort_by: 'default',
```

```
  sort_direction: 'desc',
```

```
}
```

```
const PER_PAGE = 12;
```

```

export default function EventsPage() {
  const { user } = useAuth();
  const [events, setEvents] = useState<EventType[]>([]);
  // Filters that weren't applied yet (user didn't click on search)
  const [draftFilters, setDraftFilters] = useState<EventFilters>({ ...DEFAULT_FILTERS });
  // Filters that were applied.
  const [currentFilters, setCurrentFilters] = useState<EventFilters>({ ...DEFAULT_FILTERS });
  const [meta, setMeta] = useState<PaginatedEventsMeta | null>(null);
  const [isLoading, setIsLoading] = useState(true);
  const [isLoadingMore, setIsLoadingMore] = useState(false);
  const addSnackbarMessage = useContext(SnackbarContext);

  const fetchEvents = async (filters: EventFilters, page = 1, append = false) => {
    if (append) {
      setIsLoadingMore(true);
    } else {
      setIsLoading(true);
    }

    try {
      const response = await getPosts({
        ...filters,
        friends_only: filters.friends_only ? 1 : 0,
        following_only: filters.following_only ? 1 : 0,
        page: page,
        per_page: PER_PAGE,
      });

      setEvents(current => append ? [...current, ...response.data] : response.data);
      setMeta(response.meta);
    } catch (error) {
      if (isValidationError(error)) {

```

```

    const errors = extractValidationErrors(error);
    Object.values(errors).forEach(messages => {
      messages?.forEach(message => addSnackbarMessage(message, 'error'));
    });
  } else {
    const errorMessage = extractErrorMessage(error);
    addSnackbarMessage(errorMessage, 'error');
  }
} finally {
  setIsLoading(false);
  setIsLoadingMore(false);
}
}

useEffect(() => {
  void fetchEvents(currentFilters);
}, [currentFilters])

const handleApplyFilters = () => {
  setCurrentFilters({ ...draftFilters });
}

const handleResetFilters = () => {
  setDraftFilters({ ...DEFAULT_FILTERS });
  setCurrentFilters({ ...DEFAULT_FILTERS });
}

const handleLoadMore = async () => {
  if (!meta?.has_more || isLoadingMore) {
    return;
  }

  await fetchEvents(currentFilters, meta.current_page + 1, true);

```

```
}
```

```
return (
```

```
<div className="max-w-7xl mx-auto px-4 py-8">
```

```
<div className="mb-8">
```

```
<EventSearchAndFilters
```

```
  filters={draftFilters}
```

```
  onChangeAction={({nextValue}) => setDraftFilters(nextValue)}
```

```
  onSubmitAction={handleApplyFilters}
```

```
  onResetAction={handleResetFilters}
```

```
  showSort
```

```
  disableSocialFilters={!user}
```

```
  isLoading={isLoading}
```

```
/>
```

```
</div>
```

```
{currentFilters.search === "
```

```
&& currentFilters.categories.length === 0
```

```
&& !currentFilters.friends_only
```

```
&& !currentFilters.following_only
```

```
&& !currentFilters.date_from
```

```
&& !currentFilters.date_to && (
```

```
<div className="mb-6 flex flex-col gap-2">
```

```
<h1 className="text-2xl font-semibold">Aktuālākie gaidāmie pasākumi</h1>
```

```
</div>
```

```
)}
```

```
{meta && (
```

```
<div className="mb-4 text-sm text-gray-500">
```

```
  Rāda {events.length} no {meta.total} pasākumiem
```

```
</div>
```

```
)}
```

```

{isLoading ? (
  <Loading />
) : events.length > 0 ? (
  ◇
  <div className="grid grid-cols-1 gap-6 md:grid-cols-2 lg:grid-cols-3">
    {events.map((event) => (
      <EventCard key={event.id} event={event} />
    ))}
  </div>

  {meta?.has_more && (
    <div className="mt-8 flex justify-center">
      <button
        type="button"
        onClick={handleLoadMore}
        disabled={isLoadingMore}
        className="rounded-xl border border-gray-300 px-6 py-3 text-sm font-
medium text-gray-700 transition hover:bg-gray-50 disabled:cursor-not-allowed disabled:opacity-
60"
        >
        {isLoadingMore ? 'Ielādē vēl...' : 'Ielādēt vēl'}
      </button>
    </div>
  )}
</>
) : (
  <div className="rounded-2xl border border-dashed border-gray-300 bg-white px-6 py-
12 text-center text-gray-600">
    Neviens pasākums neatbilda meklēšanai. Mēģiniet citu vietu, kategoriju vai sociālo
    filtru.
  </div>
)}

```

</div>

);

}

```
'use client';
```

```
/**
```

```
 * Šī lapa parāda pasākumus interaktīvajā kartē ar filtrēšanas iespējām.
```

```
 * Lietotājs var meklēt, filtrēt, iezīmēt interesi un norādīt dalību pasākumos tieši no kartes.
```

```
 */
```

```
import EventSearchAndFilters from '@components/Event/EventSearchAndFilters';
```

```
import type { EventFilters, EventType, User } from '@utils/Types';
```

```
import Map from '@components/Map/DynamicMarkerMap';
```

```
import { useContext, useEffect, useState } from 'react';
```

```
import { getPost, getPosts, setGoingPost, setInterestedPost, type EventResponse } from  
'@utils/post_service';
```

```
import Loading from '@components/Loading';
```

```
import EventPreview from '@components/Map/UI/EventPreview';
```

```
import { useAuth } from '@hooks/auth';
```

```
import { extractErrorMessage, extractValidationErrors, isValidationError } from  
'@utils/response_helper';
```

```
import { SnackbarContext } from '@context/SnackbarContext';
```

```
import ResponsiveOverlayPanel from '@components/ResponsiveOverlayPanel';
```

```
const DEFAULT_FILTERS: EventFilters = {
```

```
  search: "",
```

```
  categories: [],
```

```
  friends_only: false,
```

```
  following_only: false,
```

```
  sort_by: 'default',
```

```
  sort_direction: 'desc',
```

```
};
```

```
const MapPage = () => {
```

```

const { user } = useAuth();

const [posts, setPosts] = useState<EventType[]>([]);
const [open, setOpen] = useState<boolean>(false);

// Selected event
const [selectedEvent, setSelectedEvent] = useState<EventType | null>(null);
const [interested, setInterested] = useState<boolean>(false);
const [going, setGoing] = useState<boolean>(false);
const [goingUsers, setGoingUsers] = useState<User[]>([]);
const [interestedUsers, setInterestedUsers] = useState<User[]>([]);

const [draftFilters, setDraftFilters] = useState<EventFilters>({ ...DEFAULT_FILTERS });
const [currentFilters, setCurrentFilters] = useState<EventFilters>({ ...DEFAULT_FILTERS });
const [isLoading, setIsLoading] = useState(true);
const [isMobileFiltersOpen, setIsMobileFiltersOpen] = useState(false);
const addSnackbarMessage = useContext(SnackbarContext);

useEffect(() => {
  const fetchEvents = async (filters: EventFilters) => {
    setIsLoading(true);

    try {
      const result = await getPosts({
        ...currentFilters,
        friends_only: filters.friends_only ? 1 : 0,
        following_only: filters.following_only ? 1 : 0,
        per_page: 100,
      });

      setPosts(result.data);
    } catch (error) {

```



```

    if (isValidationError(error)) {
      const errors = extractValidationErrors(error);
      Object.values(errors).forEach(messages => {
        messages?.forEach(message => addSnackbarMessage(message, 'error'));
      });
    } else {
      const errorMessage = extractErrorMessage(error);
      addSnackbarMessage(errorMessage, 'error');
    }
  } finally {
    setIsLoading(false);
  }
}

fetchEvents(currentFilters);
}, [currentFilters]);

const setEventDetails = (eventResponse: EventResponse) => {
  setSelectedEvent(eventResponse.data);
  setGoing(eventResponse.meta.is_going);
  setInterested(eventResponse.meta.is_interested);
  setGoingUsers(eventResponse.meta.going_users ?? []);
  setInterestedUsers(eventResponse.meta.interested_users ?? []);
};

// After user clicks on interested or going, it will update this post
const refreshSelectedEvent = async (eventId: number) => {
  const eventResponse = await getPost(eventId);
  setEventDetails(eventResponse);

  setPosts(prevPosts => prevPosts.map(post => post.id === eventId ? eventResponse.data :
post));
};

```

```

const toggleModal = (value: boolean) => setOpen(value);
const selectEvent = async (event: EventType) => {
  setOpen(true);
  setSelectedEvent(event);
  try {
    const eventResponse = await getPost(event.id);
    setEventDetails(eventResponse);
  } catch (error) {
    const errorMessage = extractErrorMessage(error);
    addSnackbarMessage(errorMessage, 'error');
  }
}

const handleResetFilters = () => {
  setDraftFilters({ ...DEFAULT_FILTERS });
  setCurrentFilters({ ...DEFAULT_FILTERS });
}

const handleApplyFilters = () => {
  setCurrentFilters({ ...draftFilters });
}

async function handleInterested(value: boolean, eventId: number) {
  const response = await setInterestedPost(eventId, value);

  if (response.status !== 'ok') {
    addSnackbarMessage('Neizdevas atjaunot interesējas statusu.', 'error');
    return;
  }

  try {
    await refreshSelectedEvent(eventId);
  }
}

```

```

    } catch (error) {
      if (isValidationError(error)) {
        const errors = extractValidationErrors(error);
        Object.values(errors).forEach(messages => {
          messages?.forEach(message => addSnackbarMessage(message, 'error'));
        });
      } else {
        const errorMessage = extractErrorMessage(error);
        addSnackbarMessage(errorMessage, 'error');
      }
    }
  }
}

```

```

async function handleGoing(value: boolean, eventId: number) {
  const response = await setGoingPost(eventId, value);

  if (response.status !== 'ok') {
    addSnackbarMessage('Neizdevas atjaunot piedalās statusu.', 'error');
    return;
  }
}

```

```

try {
  await refreshSelectedEvent(eventId);
} catch (error) {
  if (isValidationError(error)) {
    const errors = extractValidationErrors(error);
    Object.values(errors).forEach(messages => {
      messages?.forEach(message => addSnackbarMessage(message, 'error'));
    });
  } else {
    const errorMessage = extractErrorMessage(error);
    addSnackbarMessage(errorMessage, 'error');
  }
}

```

```

    }
  }

```

```

return (

```

```

  <

```

```

    {isLoading ? (

```

```

      <Loading/>

```

```

    ) : (

```

```

      <div className="relative w-full h-full">

```

```

        {/* Shows only for mobile */}

```

```

        <div className="sm:hidden w-full">

```

```

          <ResponsiveOverlayPanel

```

```

            isOpen={isMobileFiltersOpen}

```

```

            onClose={() => setIsMobileFiltersOpen(false)}

```

```

            trigger={

```

```

              <div className="absolute left-4 top-4 z-[1100] sm:hidden">

```

```

                <button

```

```

                  className="rounded-full bg-gradient-to-r from-logo-1 to-logo-3 px-4 py-

```

```

2 text-white shadow"

```

```

                  onClick={() => setIsMobileFiltersOpen(true)}

```

```

                  aria-label="Atvērt filtrus"

```

```

                >

```

```

                  Filtri

```

```

                </button>

```

```

              </div>

```

```

            }

```

```

            title="Filtri"

```

```

          >

```

```

        <div className="mx-4">

```

```

          <EventSearchAndFilters

```

```

            filters={draftFilters}

```

```

            onChangeAction={(nextValue) => setDraftFilters(nextValue)}

```

```

            onSubmitAction={() => { setIsMobileFiltersOpen(false);

```

```

handleApplyFilters(); }}

        onResetAction={() => { setIsMobileFiltersOpen(false); handleResetFilters();
    }}

        disableSocialFilters={!user}
    />
</div>
</ResponsiveOverlayPanel>
</div>

{/* Shows for desktop */}
<div className="pointer-events-auto absolute left-4 right-4 top-4 z-[1000] mx-auto
max-w-5xl [<form]:border-0 [<form]:bg-transparent [<form]:p-0 [<form]:shadow-none
hidden sm:block">
    <EventSearchAndFilters
        filters={draftFilters}
        onChangeAction={(nextValue) => setDraftFilters(nextValue)}
        onSubmitAction={handleApplyFilters}
        onResetAction={handleResetFilters}
        disableSocialFilters={!user}
    />
</div>

<Map
    center={[56.88, 24.28]}
    zoom={8}
    markers={posts}
    className={"w-full h-full"}
    onMarkerClick={selectEvent}
/>

{/* MAP UI */}
<div className="pointer-events-auto absolute left-0 top-0 h-full">
    <EventPreview

```

```

        isOpen={open}
        toggleModal={toggleModal}
        event={selectedEvent}
        isInterested={interested}
        isGoing={going}
        interestedUsers={interestedUsers}
        goingUsers={goingUsers}
        handleInterested={(value: boolean) => selectedEvent ? handleInterested(value,
selectedEvent.id) : Promise.resolve()}
        handleGoing={(value: boolean) => selectedEvent ? handleGoing(value,
selectedEvent.id) : Promise.resolve()}
    />
</div>

{posts.length === 0 && (
    <div className="pointer-events-none fixed left-0 right-0 bottom-0 z-[1200] flex
justify-center w-full px-2 pb-4 sm:pb-6">
        <div className="rounded-full bg-white/95 px-4 py-2 text-sm sm:text-base text-
gray-700 shadow border border-brand-500 font-medium">
            Neviens pasākums neatbilda šiem filtriem.
        </div>
    </div>
)}
</div>
)}
</>
)
}

```

```

export default MapPage;

```

Layout.tsx

```
"use client";

import '@lib/echo-init'; // Initializes echo before everything else
import { Nunito } from 'next/font/google';
import '@app/global.css';
import { useAuth } from '@hooks/auth';
import Navigation from '@components/Navigation';
import { SnackbarProvider } from '@context/SnackbarContext';

const nunitoFont = Nunito({
  subsets: ['latin-ext'],
  display: 'swap',
});

const RootLayout = ({
  children
}: {
  children: React.ReactNode
}) => {
  const { user } = useAuth({ });

  return (
    <html lang="en" className={nunitoFont.className}>
      <body className="min-h-screen bg-white">
        <SnackbarProvider>
          <div className="flex flex-col h-[100dvh]">
            <Navigation user={user} />
            <main className="flex-1">
              {children}
            </main>
          </div>
        </SnackbarProvider>
      </body>
    </html>
  );
}
```

```
</body>
```

```
</html>
```

```
);
```

```
}
```

```
export default RootLayout;
```